# LLM Enhanced Task Offloading and Execution Framework in Vehicular Edge Computing

*by*

**Faysal Hossain Tomal - 221002220**
**Afzal Hossain - 221002628**
**K M Monoarul Islam Shovon - 221002213**

*Capstone Thesis (CSE 400) submitted in partial fulfillment of the requirements for the degree of*

**Bachelor of Science in Computer Science and Engineering**

**Supervisor**
**Prof. Dr. Md. Abdur Razzaque**

**Co-supervisor**
**Mr. Palash Roy**


**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**
**GREEN UNIVERSITY OF BANGLADESH**

**FALL 2025**

# DECLARATION

| | |
|---|---|
| **Title** | LLM Enhanced Task Offloading and Execution Framework in Vehicular Edge Computing |
| **Author** | *Faysal Hossain Tomal*, *Afzal Hossain* & *K M Monoarul Islam Shovon* |
| **Student ID** | *221002220*, *221002628* & *221002213* |
| **Supervisor** | Prof. Dr. Md. Abdur Razzaque |
| **Co-supervisor** | Mr. Palash Roy |

We declare that this thesis *LLM Enhanced Task Offloading and Execution Framework in Vehicular Edge Computing* is the result of our own work except as cited in the references. The thesis has not been accepted for any degree and is not concurrently submitted in candidature of any other degree.

**Faysal Hossain Tomal - 221002220**

**Afzal Hossain - 221002628**

**K M Monoarul Islam Shovon - 221002213**

Department of Computer Science and Engineering
**Green University of Bangladesh**

**Date:** January 30, 2026

# CERTIFICATE OF ENDORSEMENT

This is to certify that the thesis *LLM Enhanced Task Offloading and Execution Framework in Vehicular Edge Computing* by **Faysal Hossain Tomal, Afzal Hossain & K M Monoarul Islam Shovon** has been successfully defended and examined.

Based on the evaluation and discussion held on **January 30, 2026**, we, the undersigned, hereby endorse the thesis for acceptance and approval.

---

**Prof. Dr. Md. Saiful Azad**
**Board Member, Board: 01**

Dean
Faculty of Science And Engineering
Green University of Bangladesh

---

**Sabbir Hosen Mamun**
**Board Member, Board: 01**

Lecturer
Department of Computer Science and Engineering
Green University of Bangladesh

---

**Md. Moinul Islam**
**External Member, Board: 01**

Head of Software Development & Implementation
Synesis IT Ltd

**Date:** January 30, 2026

Department of Computer Science and Engineering
Green University of Bangladesh
Purbachal American City, Kanchan, Rupganj, Narayanganj-1461, Dhaka, Bangladesh

# CERTIFICATE

This is to certify that the thesis **LLM Enhanced Task Offloading and Execution Framework in Vehicular Edge Computing**, submitted by **Faysal Hossain Tomal** (Student ID: 221002220), **Afzal Hossain** (Student ID: 221002628) & **K M Monoarul Islam Shovon** (Student ID: 221002213) an undergraduate student of the **Department of Computer Science and Engineering** has been examined. Upon recommendation by the examination committee, we hereby accord our approval of it as the presented work and submitted report fulfill the requirements for its acceptance in partial fulfillment for the degree of *Bachelor of Science in Computer Science and Engineering*.

**Prof. Dr. Md. Abdur Razzaque**
**Chairman**

Department of Computer Science and Engineering
University of Dhaka

**Mr. Palash Roy**
**Lecturer**

Department of Computer Science and Engineering
University of Dhaka

**Syed Ahsanul Kabir**
**Associate Professor & Chairperson**

Department of Computer Science and Engineering
Green University of Bangladesh

**Place:** Purbachal American City, Kanchan, Rupganj, Narayanganj-1461, Dhaka, Bangladesh
**Date:** January 30, 2026

# ACKNOWLEDGMENTS

Alhamdulillah, First and foremost, we are deeply grateful to the Almighty for allowing us to get this opportunity from the **Green University of Bangladesh** for providing opportunities and resources to continue our educational and research efforts. We extend our heartfelt praise for the **Department of CSE** for promoting a learning environment that promotes intellectual development and innovation.

We especially thank our research supervisor **Prof. Dr. Md. Abdur Razzaque Sir**, **Chairman, Department of CSE, University of Dhaka** and co-supervisor **Mr. Palash Roy Sir**, **Lecturer, Department of CSE, University of Dhaka** for granting us the opportunity to work under their invaluable guidance. Their insight, inspiration, and continuous encouragement have been instrumental at every stage of this research. It has been a great privilege and honor to work under their supervision.

We would like to express our honest thanks to **Syed Ahsanul Kabir Sir, Chairperson of the Department of CSE, Green University of Bangladesh** for continuous support and visionary leadership on our educational journey.

Our deepest thanks to all the respected faculty members of the department. His dedication to teaching and mentorship has contributed significantly to the development of our knowledge, skills, and professional approaches.

We are always grateful to **our parents and siblings** for their unconditional love, prayer, and unbreakable support. Their victims have provided the basis for our education and personal development.

We also want to acknowledge the contribution of our senior **Md. Abdullah Al Sami**, whose collaboration, dedication, and teamwork made the journey smooth and more fun.

Special thanks to our friends and well-wishers for their continuous support, courage, and companionship throughout this research journey. Finally, we extend our heartfelt gratitude to everyone who supported us directly or indirectly in successfully completing this research.

*Faysal Hossain Tomal*, *Afzal Hossain* & *K M Monoarul Islam Shovon*
Department of Computer Science and Engineering
**Green University of Bangladesh**
**Date:** January 30, 2025

Dedicated to the Almighty for strength beyond my own, to my parents for unwavering sacrifice, to my little sister as my reminder of hope, and to my countless sleepless nights along the way.

*– Faysal Hossain Tomal*

Dedicated to myself for perseverance, to my parents for unlimited support, to my teachers for guidance, and to my friends for constant encouragement.

*– Afzal Hossain*

Dedicated to my parents for their endless love, to my honorable teachers for their wisdom, and to my friends for walking beside me throughout this journey.

*– K M Monoarul Islam Shovon*

# ABSTRACT

Vehicular Edge Computing (VEC) has become a key enabler for intelligent transportation systems by providing low-latency task execution and resource efficiency for safety, navigation, and infotainment applications. Along with the ever-increasing production of tasks and delay-sensitive applications from connected vehicles, efficient management of computation tasks at the network edge is quite vital. Although, due to the restriction of computing power in vehicle onboard units, dynamic mobility, varying network conditions, and complicated edge resource management are factors that prove optimal task offloading is a challenging issue for VEC. The state-of-the-art works have mostly adopted reinforcement learning (RL) or static optimization that reasons on numerical states of systems towards task semantics, contextual urgency, and flexible prioritization, but such methods are incapable of adapting to the real-time scenarios of highly dynamic vehicular environments. To address these issues, in this thesis we present LEVEC, a context-aware Large Language Model (LLM)-Enhanced VEC framework that incorporates LLM-based reasoning into task prioritization, scheduling, and offloading. Our proposed LEVEC framework leverages contextual task information, including task type, urgency, deadlines, and real-time system status, to produce priority scores to decide adaptive local, partial, and full task execution over RSUs while jointly optimizing latency and cost. Experimental results demonstrate that compared to the state-of-the-art solutions, including MAPTD3 and LLM-DT, LEVEC consistently achieves significantly lower latency with an average reduction of approximately 50.9% against MAPTD3 and 54.7% against LLM-DT and superior service cost efficiency, yielding an average cost saving of about 12.6% relative to MAPTD3 and 35.5% relative to LLM-DT. These improvements are particularly pronounced under varying vehicular densities, thereby confirming the effectiveness of LLM-based context-aware decision-making for scalable real-time vehicular edge computing systems.

**Keywords**: Vehicular Edge Computing (VEC), Large Language Model (LLM), Reinforcement Learning (RL), Task Prioritization, Task Offloading, Latency, Cost, Context, LEVEC, MAPTD3, LLM-DT.

# Contents

# List of Figures

# List of Tables

# 1. Introduction

## 1.1 Background

Vehicular Edge Computing (VEC) [1] is a type of technology that combines edge computing with vehicles, and vehicles can offload some computational tasks to nearby edge servers or RSU. Even the idea of edge computing started to gain attention in the early 2010s as a way to cut down latency and network bottlenecks caused by cloud-based computations where data needs to travel back and forth from the source to the data center. The rise of the "edge computing revolution" has been important in the use cases of IoT, telecoms, and smart cities.

To meet the increasing demands of vehicular networks by 5G, VEC technology was almost the only choice for low-latency and high-throughput communication. Integrating V2X communication with VEC systems enables vehicles to communicate among each other and infrastructure such as RSUs, thus improving the efficiency of task offloading and resource allocation.

Important events that helped shape VEC are the deployment of 5G networks and edge computing systems at the network's edge. These developments allowed vehicles to utilize neighboring RSUs for computation offloading, i.e., time-consuming tasks (e.g., collision detection or real-time navigation updates) would be extensively handled by not sufficiently equipped vehicles.

One significant progress that has been made in the area is enabling task offloading policies to lift utilization of local as well as remote computing resources up. With the adoption of AI and ML, these tactics were enhanced for smarter task scheduling and resource management. AI-based model integration, such as Large Language Models (LLMs) [2], opened new doors in context-aware task prioritization, hence aiding vehicles in taking more intelligent decisions in relation to task execution.

According to Precedence Research [3], in figure 1.1, we can see the rapid growth of the LLM market as well as the growing reliance on these technologies. The global LLM market is estimated to reach USD 149.89 billion in 2035, rising from USD 10.57 billion in 2026 with a CAGR of 34.44% from 2026 to 2034, as per recent estimates. This trend of LLM market size growth witnesses the growing significance of LLMs to task

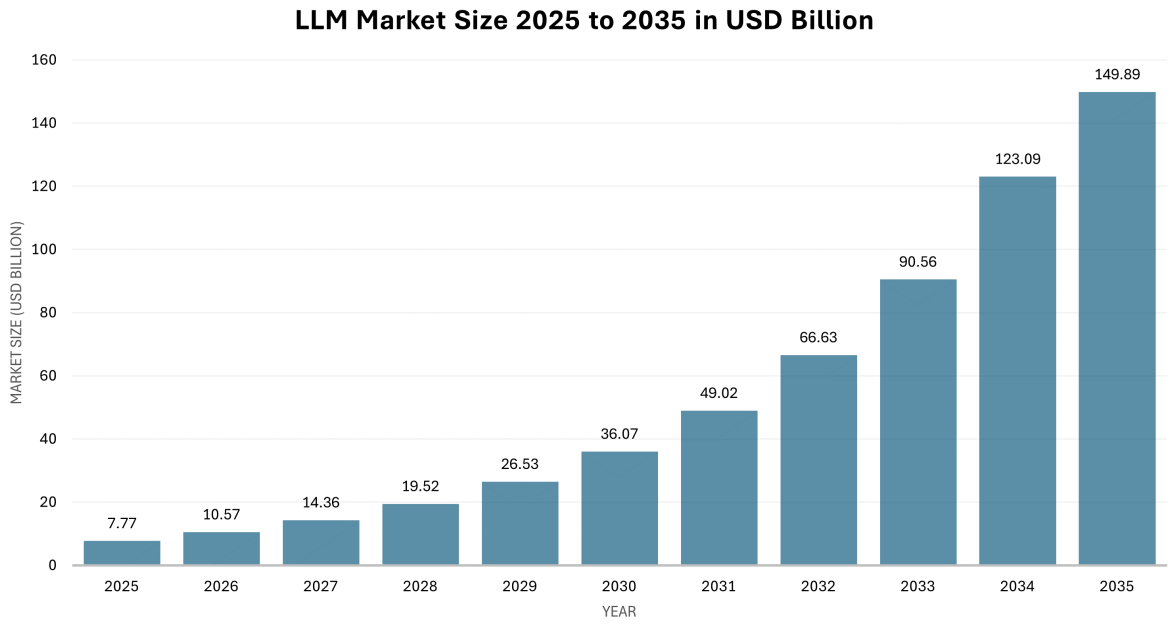



Figure 1.1: LLM Market Size from 2025 to 2035 in USD Billion

offloading and optimization in VEC systems, rendering them a fundamental enabler for intelligent vehicular networks.

Currently, VEC is a crucial technique for both autonomous vehicles and intelligent transportation systems, which ask for low-latency, high-efficiency execution of tasks to guarantee the interests of safety-critical applications. The incorporation of the LLM-based models, as introduced by us in this research, extends even more the functionalities of the offloading and execution infrastructure by providing to the vehicles the ability to statically assign a context-dependent priority per task.

### 1.1.1 Vehicular Edge Computing (VEC)

Vehicular Edge Computing (VEC) refers to the integration of edge computing in the vehicular infrastructure. From figure 1.2, we can see the aim of this approach is to shift data processing from vehicles to nearby Roadside Units (RSUs), which are distributed along the roads or other proximal points. These systems are introduced due to the importance of real-time, safety-critical tasks like automatic driving, navigation, and collision avoidance, which require low-latency communications.

In conventional cloud computing, data collected from vehicles are transferred to a central data center for further processing, and due to this physical distance between the vehicle and the cloud, there is a higher latency in communication. This type of delay is not necessarily desirable in tasks that need to make real-time decisions. For instance, an autonomous vehicle needs to respond almost immediately once a signal pops up.

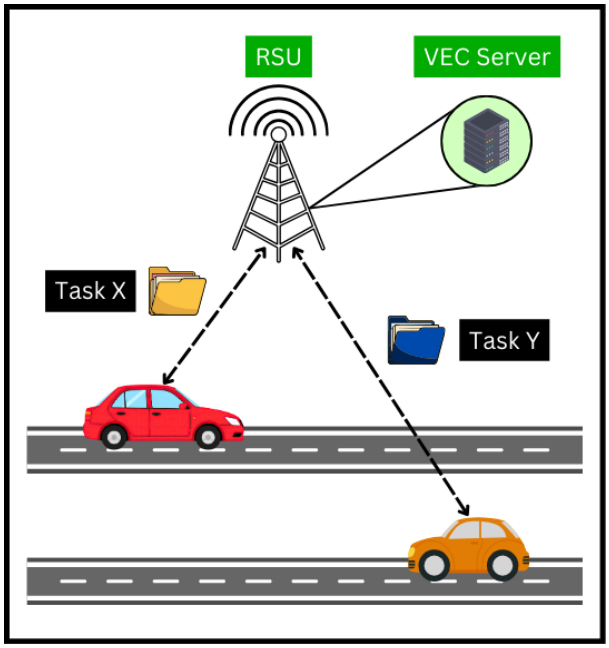



Figure 1.2: Vehicular Edge Computing

Recent work on VEC attempts to address these problems by pushing the computation out toward the vehicles. This enables the real-time fulfillment of tasks requiring sensor data processing and V2I communication as well as route planning and enhances the performance of vehicular systems. VEC is based on the trial time and an edge server, which is typically accompanied by the RSUs and can help conduct in-device processing, achieving load balancing via RSUs as well. The system provides effective computation for urgent tasks, such as safety-critical ones, with less delay, while efficiently serving other regular tasks with flexibility.

VEC is also central to autonomous driving and building smart cities, where vehicles communicate with the the city's infrastructure to facilitate smart traffic management and enhance overall safety. For example, VEC can be used for dynamic traffic lights control according to the car density or an emergency vehicle routing in a disaster. This high level of integration leads to even smarter, more intuitive and powerful cities.

#### 1.1.1.1 Evolution of edge computing in vehicles

Edge computing [4] originated in the early 2000s to address increasing demands for low-latency, high-throughput processing on distributed networks. Originally, edge computing was developed and used in the Internet of Things (IoT) [5] and telecommunications, but it became necessary to process data localized at the car vehicle level due to the increasing use of autonomous vehicles.

Traditionally, vehicles were more dependent on cloud computing for processing functions such as vehicle navigation and safety. Cloud systems were powerful, but they

weren't optimal for making split-second decisions in real time when it comes to self-driving vehicles, where milliseconds matter. Due to the dependency on cloud-based services, they suffered from high latencies and congestion, particularly in environments where vehicles need to act instantly.

The adoption of edge computing in vehicles served as a solution for these problems, since the processing load could be distributed across RSUs and edge servers that are deployed at strategic places such as over the road. This enables quicker, more efficient computation of data in real time and minimizes long-distance data transmission demands, thus instant decision making.

With the application of 5G networks, VEC has become even more practical in vehicular networks. The higher transmission rates help in real-time information sharing between VEC, RSUs and cloud-level computing infrastructure are leading to a further improved performance of the VEC systems.

#### 1.1.1.2 VEC system architecture

VEC consists of three functional components [6] [7] that interact with each other to ensure the vehicular tasks are offloaded efficiently and executed in real time.

1. **Vehicles:** Vehicles have on-board computing units for processing local tasks (e.g., sensor data from cameras or LiDAR). For tasks that demand higher computation than what the vehicle is capable of, they are offloaded to proximate RSUs or edge servers.

2. **Road-side Units (RSUs):** RSUs are installed along roads that act as local processing points for vehicles. They come with edge servers that can perform diverse kinds of computation, such as traffic management and vehicle safety surveillance. RSUs are linked to cars as well as centralized cloud platforms for effective communication and task execution.

3. **Edge Servers:** Edge servers execute tasks that have been offloaded from vehicles. These servers are in RSU or other strategic locations so that higher-priority tasks are scheduled with as little delay as possible. The VEC system is mainly composed of the vehicle, RSU and edge server interaction, which can provide efficient data processing and task offloading.

This architecture ensures that the computational load of the tasks by the vehicles is properly distributed in a way that maximizes task processing with minimal latency efficiently, which makes it suitable for real-time decision making vehicular tasks.

### 1.1.2 Task Offloading and Execution in VEC

Task offloading is an important part of VEC and it refers to the offload of computational tasks from vehicles towards RSUs for processing. This is particularly critical for high-computational tasks.

The decision to offload is usually made according to the computational needs of the task, available resources, and network conditions. By outsourcing computation, V2I can alleviate the burden on power-constrained vehicles and help forward high-priority messages with less congestion delay.

#### 1.1.2.1 Traditional task offloading & execution

In traditional cloud computing, the task offloading is achieved by transmitting data to remote cloud data centers for processing. This method is effective for non-time-sensitive applications, but it leads to high end-to-end delays because of the long distance between the vehicle and cloud, thus making it unsuitable for real-time vehicular services. For instance, autonomous vehicles may suffer from latencies in reacting time due to high processing when the system data is off-loaded to the cloud.

Classical static offloading strategies, which do not consider vehicle dynamics in terms of speed or network flow are used in conventional architectures. This will bring about ineffective use of resources, processing delay, and poor performance of the system as a whole.

#### 1.1.2.2 Task offloading in VEC & execution

For task offloading in VEC, computational tasks are offloaded to nearby RSUs in order to minimize the amount of data that needs to be transferred over long distance than sending data to cloud. This procedure allows the data to process in real time while enabling fast decisions, which is vital for modern days.

The task management framework of VEC guarantees the tasks to be scheduled on the most appropriate resource according to task priority and system capacity. For example, instead of relying on whatever order the tasks are ordered in memory (sorted randomly), safety-critical tasks are given more importance than non-safety-critical tasks.

The VEC has an architecture where tasks can be partially offloaded, splitting a task into sub-tasks and having some of those processed in the vehicle and others offloaded to edge servers for further processing. This hybrid offloading mechanism achieves efficient resource utilization and has bounded serving latency for high-priority demands.

### 1.1.3 Traditional Task Offloading vs. Context-Based Task Offloading

The difference between [8] traditional task offloading and context-based task offloading is in how we assign tasks to resources. Traditionally, the decision of task offloading is simply based on a fixed rule, like the size or type of the task. However, these models fail to consider dynamic factors such as network state and task urgency during the estimation process, leading to reduced performance in real-time vehicular networks.

In contrast, context-aware task offloading requires decision-making in real-time, and decisions are based on current status (e.g., speed of the vehicle, bandwidth of the network, urgency of the task). This helps give more priority to critical tasks than non-critical ones.

#### 1.1.3.1 Core difference between traditional and context-based

The main difference between traditional task offloading and context-based offloading is the flexibility of the decision-making process. The static models come with the limitation that when a task is generated, it will always be offloaded using fixed rules. Traditional offloading is not very flexible for real-time applications whose contending factors, such as network load and vehicle mobility can change frequently.

In contrast, context-aware task offloading is dynamic. The system action is not computed dynamically, since it takes real-time variables (e.g., vehicle's position, resources available, and network condition) into account before making decisions about offloading tasks. As a result, there is an increased resource sharing and a better system efficiency.

#### 1.1.3.2 Advantages of context-based task offloading

The advantages of context-aware task offloading are significant. Firstly, it is a great approach, as decisions are never made with old information. For example, safety-critical functions can be given precedence if the network is congested and processing delays may occur. Secondly, it saves resources by adjusting to variable conditions. For instance, if the vehicle is in a low-bandwidth environment, more computations can be done locally in order to reduce the computational costs.

Considering the task urgency level, vehicle speed and network status, context-based task offloading ensures that high-priority tasks are processed first, which can mitigate the instance of autonomous driving system delay and enhance overall system responsive.

### 1.1.4 Diverse Applications of Vehicular Edge Computing

VEC could revolutionize numerous industries and applications by offering faster and more efficient data processing capabilities for vehicles and infrastructure. Some of the critical applications, like autonomous vehicles, traffic management and urban infrastructure, require the real-time decision-making that is improved by VEC, since it is capable of outsourcing computation off mobile devices to edge servers or RSUs. The main contribution of VEC is to mitigate the latency and bandwidth challenges in conventional cloud-based solutions.

- **Autonomous Vehicles:** One of the most notable VEC applications is in autonomous vehicles [9]. These cars use multiple sensors and data such as radars, LIDARs, cameras, GPS systems. But the vehicles require considerable computing power to work with that data in real time. The edge computing in the VEC approach provides the required processing capabilities by offloading part of it to RSUs or edge servers closer to the vehicle. For instance, tasks such as pedestrian detection and collision avoidance need to be processed instantly. This time-consuming processing can be directly performed in the edge computing node so that the vehicle can almost immediately respond to road situation variations, improving both safety and efficiency.

- **Traffic Management and Smart Cities:** Another great application of VEC is in building smart cities. Traffic control systems [10] can be one of those in which the load is carried by traffic lights, sensors and monitoring systems to nearby RSUs. This makes traffic flow and signal timings readily adaptable in real time, minimizing the congestion and enhancing the safety of traffic. Furthermore, VEC also enriches the computational power supports for smart parking systems, environmental monitoring and city-wide surveillance. Sensored vehicles can make real-time contributions to urban infrastructure, which may be capable of improving cities more effectively and responsively.

- **Emergency Response Systems:** VEC also has an important role in the domain of emergency response systems [11] with time-critical data. For instance, in an occurrence of a health crisis, ambulances that are on-board with sensors and communication devices can forward vital information (such as ECG readings and body temperature) of a patient to the nearby RSUs for processing. This allows healthcare providers to get up-to-the-moment information about a patient's condition and prepare for what needs to happen when they come in. Fire engines and police cars, by the same token, can use VEC to get real-time feedback about emergency avenues or traffic disruptions or dangerous situations.

- **Industrial and Commercial Applications:** In the industry, VEC can be applied to fleet management and logistic optimization [12]. For example, trucks can maintain an open communication with RSUs to get real-time information about traffic conditions, plan routes, and receive vehicle diagnostics. Fleet managers also can monitor the performance of their individual vehicles and make real-time adjustments, such as fuel efficiency or maintenance requirements, using VEC systems. Furthermore, due to the VEC, the schematic factory can benefit from utilizing connected vehicles for the transport of materials in a more efficient way and with real-time management of the supply chain.

- **Remote Monitoring and Other Applications:** VEC's local processing capabilities are also well suited to remote monitoring systems [13]. For instance, environmental sensors located in cars can analyze air quality, traffic noise, or weather conditions and the data could be sent to local edge servers, which are capable of processing and analyzing the information for instantaneous action. Likewise, construction vehicles and mining machinery can also apply VEC to monitor machine status for optimum workflow and minimum downtime.

### 1.1.5 Large Language Models (LLMs) in VEC

Large Language Models (LLMs) have attracted a lot of interest lately for their capabilities in natural language processing (NLP), text generation and contextual understanding. Models like GPT-3 and Llama-3 have been used in real-world applications across industries ranging from healthcare to finance, as well as customer service. In VEC, LLMs can help the decision-making task, especially in task offloading, prioritization of tasks, and dynamic resource allocation.

- **Overview of LLMs:** LLMs are trained on large-scale data (mainly textual data) and can understand as well as produce human-like language. These models are capable of learning deep contextual relationships between words, phrases, and paragraphs, making them extraordinarily powerful in cases when the context is essential to understanding. Although LLMs were originally developed for applications like chatbots and automatic content generation, their potential for enhancing task offloading for VEC systems has received more attention recently.

- **Role of LLMs in Task Offloading in VEC:** In VEC, task offloading decisions are likely to take into account various factors like task urgency, vehicle speed, network situation, and available computing resources. In traditional offloading systems, predefined rules and static thresholds are used to determine where tasks should be executed. However, such models are mostly inflexible and do not take the dynamic nature of vehicle environments into consideration. Context processing

and numerical task metadata, as well as natural language descriptions of a task, are the features that can go beyond LLMs for task off-loading systems. This allows smarter, more agile decision-making that prioritizes the most important tasks in real-time, even if the situation changes while execution is still ongoing.

For instance, an autonomous car can produce multiple tasks to respond to navigation updating processing, sensor data analysis, and responses to infotainment-related requests. A conventional model for offload scheduling might rank these tasks by task size or type, while an LLM-extended model might assess a context based on natural language (such as "need to emergency brake" and "approaching intersection") to prioritize time-critical/safety-critical tasks in order to prevent high-priority tasks from being processed with unacceptable delay.

- **Scope of LLMs in VEC:** The application of LLMs in VEC is not only limited to task offloading [14] from the mobile devices but also for aspects such as resource management, task processing, and system improvement. For example, an LLM can predict the load of RSUs and distribute the tasks in a more compact way, preventing overloading the specific resource at its maximum capacity. LLMs optimize the allocation of computing resources by analyzing historical data and real-time traffic state, thus ensuring that tasks are offloaded to the most suitable RSUs or edge servers as per current network and resource availability.

  In addition, LLMs may serve to enhance the communication between vehicles and RSUs by a more context-aware interaction. For instance, vehicles can issue task requests in natural language form (e.g., "request process to safety-critical tasks near the intersection") and obtain responses that reflect the current states of available resources, traffic conditions, and network load. This would enable rapid decisions (e.g., in real-time) to be made without the need for a complex environment with dynamic readiness.

  Other than task offloading and resource assignment, LLMs can be applied to predictive analytics, where they process historic data of many vehicles and sensors for predicting future conditions or traffic. Such predictions might facilitate a proactive control of traffic, route planning for vehicles, and coordination of emergency services, as well as an additional improvement of the overall performance for VEC systems.

### 1.1.6 State of the Art in the Modern Context

Nowadays, VEC is undoubtedly the most discussed research area in autonomous transportation systems and intelligent transportation networks. Integration of edge computing in vehicular networks enables vehicles to outsource their computational

tasks to Roadside Units (RSUs) or edge servers, slowly removing the dependence on cloud infrastructure as well as reducing computation delay. Moreover, the introduction of 5G has vastly improved the capability to enforce real-time low-latency communication between vehicles and edge infrastructure recently, which is very important for safety-critical applications of vehicles such as collision avoidance and real-time navigation.

However, the investigation on how to optimize the task offloading in VEC systems is being researched day by day. In recent times, work has attempted to create algorithms that aim at a tradeoff between task delay and cost in such a way as to improve resource efficiency. For example, machine learning algorithms are being adapted into task offloading frameworks to model network state and take optimal offloading decisions while the vehicle is on the move. These models estimate available bandwidth, vehicle mobility, and RSU resources, which can be used for context-aware decision-making.

One such interesting emerging tool is applying Large Language Model (LLM)-based methods for priority and offloading decisions. Models such as Llama have been looked into, utilizing their decisions based on natural language content in task descriptions that can also enhance the accuracy of task scheduling. LLMs in task management systems can be used to make more informed and adaptive decisions that improve the fairness and efficiency of resource allocation.

There are, however, several technical issues that need to be addressed, such as the bandwidth limitation, which is still problematic, particularly for sparse RSU regions. Also, the mobility of vehicles at the network edge means network conditions are hard to predict in real time, which ultimately complicates offloading strategies. Moreover, privacy concerns also occur for vehicle-generated data whose content is potentially sensitive, such as driver behavior and vehicle location. Real-world deployment of VEC frameworks also encounters practical challenges like including system scalability and existing infrastructures and new technologies integration.

The latest research keeps on trying to address these limitations to improve the robustness of VEC systems and their adaptability in real-life applications.

### 1.1.7 Importance and Relevance of the Topic

VEC has gained increasing attention recently as optimal task offloading is crucial. Vehicles may get significant benefit in performance and efficiency from offloading their computationally complex tasks to process over nearby RSUs, especially in the real-time scenarios. For example, in autonomous vehicles, low-latency processing is very critical to the performance of safety-critical tasks such as collision avoidance, lane detection, or pedestrian detection. We can improve the performance of these tasks by employing

task offloading, which can lead to more effective and dependable executions, as well as improved overall safety.

On a larger scale for the society, deployment of VEC will result in smart cities, which may allow reduced traffic jams and air pollution (due to vehicles) as well as better road safety based on enhanced V2I communication. Moreover, LLMs can offer real-time modelling of the context for task prioritization to better manage resource allocation, processing higher-quality tasks with higher priority, including safety-critical ones.

In addition, the economic benefit from task offloading is significant. Automotive industries can reduce their operation cost in computation tasks, which contributes to profit by reducing latency and cost of the vehicular system. Furthermore, there may be new use cases where the introduction of VEC systems will lead to the creation of new possibilities like connected vehicles, intelligent transport system (ITS), and autonomous fleets which are not widely used yet today but capable of fostering innovation and economic growth within the transport sector tomorrow.

For individuals and communities, solutions developed in this research can allow real-time traffic safety that may ultimately improve roadway efficiency and mobility on the part of individuals, particularly in urban areas subject to extensive congestion and delay. Besides, efficient offloading of tasks contributes to the energy savings due to that vehicles consume less fuel with the most resource-efficient solutions for executing tasks for the communities involved by the individuals.

By addressing the issues related to VEC and task offloading, this research will lead to the development of autonomous systems and smart cities that bring about benefits to both people and industry, making our world a safer, more effective and sustainable place.

## 1.2 Motivation

Vehicular Edge Computing (VEC) faces various obstacles because of the limited computational capabilities of onboard units of the vehicles and the complexity of the edge server resource management following task features. As tasks are generated by vehicles continuously in the vehicle network, the large amount of tasks is hard to be scheduled according to their importance in highly dynamic mobile environments. The correct prioritization also becomes difficult due to a high incoming rate of tasks and rapidly varying system conditions such as mobility, network load, and application demands. It is still a challenge to handle such heavy task applications while minimizing system latency and cost and making context-aware decisions in a real-time way in VEC. Reasoning ability Large Language Models (LLMs) can be leveraged to tackle these challenges by promoting task processing and resource management based on logic

and context, thereby facilitating more intelligent and flexible strategies in the dynamic vehicular environment.

There are several recent works on VEC task offloading and resource allocation. The study in [15] proposed a heterogeneous VEC architecture incorporating Task Vehicles (TaVs), Service Vehicles (SeVs) and RSUs, and the PG-MRL technique, which jointly optimizes the decision for offloading and resource allocation leveraging potential games along with multi-agent DDPG. Although this approach reduced system delay, it relied heavily on significant reinforcement learning training and did not explicitly consider task context such as deadline sensitivity or changing network conditions during runtime. In [16], a vehicle–road collaborative edge computing network was studied and the authors developed a hybrid hierarchical deep reinforcement learning (HHDRL) to solve delay cost and energy consumption minimization. Nevertheless, their method suffered from heavy computational complexity associated with the hierarchical decision trees and was not suitable for dynamic vehicular environments. Another recent work [17] presented a D4PG-based offloading and resource allocation methodology that predicts future channel state information and dynamically allocates resources to reduce execution delay. While time-varying channels and resource dynamics are taken into account in this approach, the offloading decisions and resource allocation have not been jointly considered together with semantic or contextual task awareness. These limitations of the prior work lack of real-time contextual awareness, complex algorithmic structures, and separation of decision components have inspired us to investigate a new framework that exploits Large Language Models (LLMs) for context-aware task prioritization and decision-making and thus improves the effectiveness of adaptive VECs on the fly.

## 1.3 Problem Statement

VEC systems are being considered as a solution to mitigate the deficiencies of vehicle onboard computational resources by offloading resource-consuming tasks to RSUs in the vicinity. But VEC also brings some challenges, e.g., task offloading and resource management are issues to be addressed. They made the task prioritization and offloading decision-making very difficult due to dynamic settings of vehicular environments such as network variations, vehicle mobility, and computational demand changes. It is still an unsolved problem to guarantee the costly related task deadlines and prevent the waste of system resources.

Furthermore, existing approaches in VEC fail to utilize context information sufficiently, including the speed of vehicles, urgency of task requests, and network condition, all of which may have a strong impact on task priority ordering and load offloading. However, dynamic factors are frequently neglected by the existing methods, as no full

solution is offered for real-time and context-aware decision-making in task execution. Hence, an intelligent task offloading and resource management system in VEC that makes the data-driven, context-aware decisions is highly essential.

The main research questions driving this thesis are as follows:

- How can LLMs be integrated into the task offloading and execution process in VEC to dynamically prioritize tasks based on contextual metadata such as task urgency, network conditions, and vehicle-specific parameters?
- How can the computational resources of vehicles and RSUs be optimally managed with task offloading in highly dynamic vehicular environments while minimizing latency and cost, and ensuring that high-priority tasks meet their deadlines?
- How does the priority queueing assist in dynamically scheduling vehicular tasks in a VEC environment, and what role does it play in optimizing the decision-making process for local task execution versus task offloading to RSUs?

## 1.4 Research Objectives

The main goal of this research is to develop a smart task offloading framework for VEC systems that leverages LLM models for dynamic context-based prioritization and resource allocation. This framework is designed with the aim of tackling a dynamic vehicular environment by considering variable network conditions, vehicle mobility, and resource restrictions in terms of available computation capacity in order to optimize task execution with low latency at reduced cost.

The specific objectives of this research are as follows:

- To integrate LLMs into the task offloading and execution process in VEC, enabling dynamic prioritization of tasks based on contextual metadata such as task urgency, network conditions, and vehicle-specific parameters.
- To develop an optimization model that manages the computational resources of vehicles and RSUs, ensuring efficient task offloading in dynamic vehicular environments while minimizing latency and cost, and meeting task deadlines.
- To explore and implement a priority queueing model to dynamically schedule vehicular tasks, optimizing the decision-making process for local task execution versus task offloading to RSUs.

## 1.5 Research Contributions

In this research, we have introduced several key contributions to the field of VEC, particularly in task offloading and execution with the implementation of LLM. As this solution is a new approach to this existing problem, this research has some key contributions, which are outlined below:

- This research introduces a new foundation for the integration of LLMs in the improvement of offloading decisions in VEC systems. Leveraging real-time contextual information (i.e., the task urgency, network situations, and vehicular mobilities), the proposed framework can make more informed decisions (e.g., enhanced latency-efficient task execution).
- The proposed offloading method is compared in detail with traditional offloading models. Contrary to standard systems based on static rules, or cold-path parameters, the context-based model adjusts automatically to the instantaneous situation (vehicle speed, network load, and input task) and offers a better task execution performance as well as resource usage.
- Our research also presents an enhanced task execution model based on edge computing to address processing efficiency. By dividing tasks into smaller components that can be executed in a distributed manner, including on-vehicle and edge server, it guarantees the low-latency execution of safety-critical tasks and relieves the overload for the vehicle's onboard system.
- In addition, our study improves resource allocation in VEC systems through dynamic context-awareness. By constantly tracking the status of vehicles, network state, and real-time task requesting, the model dynamically adjusts resource assignment policy and scheduling strategy to schedule tasks to be processed in proper resources with a view that overall system efficiency is maximized while processing time delay can be minimized.

## 1.6 Thesis Outline

In this dissertation, the remaining chapters are organized as follows:

- **Chapter 2,** ***Literature Review***, discusses the existing research related to context-aware task offloading and execution. It also includes a comparison of the existing works as well as the current solutions to the stated problem. This chapter shows the current works and the literature gap which is addressed by our work.

- **Chapter 3, *Methodology*,** describes our context-aware LLM-enhanced task offloading and execution framework. This chapter includes a detailed exposition of our proposed solution and how they operate in our proposed solution.
- **Chapter 4, *Results and Discussion*,** shows the performance evaluation of our proposed solution and a comparative analysis of the experimental results compared to the state-of-the-art models, accompanied by graphs to evaluate our solution in action.
- **Finally, Chapter 5, *Conclusion and Future Work*,** concludes with the key findings of our research as well as outlines our work's limitations and the scope for future work and potential improvements to the solution.

# 2. Literature Review

## 2.1 Introduction

With the massive growth in intelligent transportation systems and autonomous vehicles [18], it becomes more urgent to address the problem of task offloading and resource allocation in vehicular networks. The use of VEC is becoming a vary lucrative solution for the processing needs of vehicles, especially for real-time and safety-critical applications. This vehicular offloading mechanism allows vehicles to hand over tasks from themselves to the nearby RSUs or edge servers in order to decrease the task's latency and bandwidth congestion and to guarantee that vehicles can rapidly process compute-intensive tasks.

Although VEC offers some promising ability, there are still some challenges to overcome. As vehicles drive through dynamic environments, they experience diverse network conditions, movements and resources. Effective decision-making on task offloading should take into account a range of considerations such as computational demand, the urgency of a task to be processed or passed back to a control center, network bandwidth and the local position and velocity of the vehicle at runtime. Moreover, as we adopt LLMs to prioritize tasks based on computed scores, context-aware decision-making gains more importance in deciding how the task should be performed and offloaded.

While deployment of VEC systems could address many of these issues, several challenges arise regarding load balancing over multiple RSUs and fair allocation of resources between competing vehicles. The optimal resource allocation for such a dynamic, heterogeneous system continues to be challenging, and despite many advances, there is still great demand for adaptive models that can adapt to changing scenarios at runtime. This everlasting challenge motivates the continuous deployment of task offloading methods, resource allocation schemes, and LLMs incorporation that improves decision-making in vehicular edge networks.

## 2.2 Literature Review

This section reviews the state-of-the-art current research contributions addressing task offloading and execution models in VEC with a focus on context-aware prioritization and LLM integration.

### 2.2.1 Adaptive Resource Allocation in IoV

The paper [19] investigates the resource allocation problem for MEC in the IoV. It proposes a new adaptive joint resource allocation mechanism, aiming at maximizing system performance regarding capacity, service delay, and energy consumption in highly dynamic IoV surroundings. The solution is based on the use of Deep Reinforcement Learning (DRL) to realize real-time adaptive assignment for uplink, computing, and downlink resources, while coping with challenging tasks such as task offloading, system capacity limits, and energy efficiency problems.

#### 2.2.1.1 Methodology

The authors showed the resource allocation problem as the multi-actor, multi-resource optimization problem in dynamic IoV environments. In response to this issue, they present a multi-actor parallel twin delayed deep deterministic policy gradient (MAPTD3) algorithm for resource control in the system. They employed a task-slicing technique to specify the state space and reward function for the DRL agent so that it can autonomously utilize system resources without knowing in advance how the environment behaves. The proposed approach makes it possible to adapt replenishment strategies at runtime, according to the dynamics of a system and tasks model, increasing the readiness of reaction on real IoV challenges.

#### 2.2.1.2 Contributions

In the paper, the authors proposed a novel adaptive resource allocation approach, which incorporates DRL into communication and computing resource optimization in IoV. The main contributions are a task slicing approach, which translates dynamic IoV tasks into the state space and reward function of the DRL agent. The authors also propose the MAPTD3 algorithm for efficiency improvement with space complexity reduced. Extensive simulations were conducted to validate the proposed scheme and it is demonstrated that compared with traditional resource allocation strategy, their proposed algorithm can offer significant benefits in system capacity, energy consumption, and task service delay in high-dynamic environments.

#### 2.2.1.3 Limitations

The paper admits that, in spite of the remarkable improvements achieved by the MAPTD3 algorithm on system performance, it remains challenging to address complex optimization of the resources under highly dynamic environments. Actually, the developed system does not completely consider the interaction between various factors such as QoT in MEC environment and may cause the ineffectiveness in some practical contexts. Furthermore, the architecture does not address how to incorporate advanced methods (e.g., reinforcement learning-based task prioritization), which could improve the efficiency of executing behaviors under critical conditions. These areas represent potential future work that could help the system generalize adequately to more complex and rapidly changing environments.

### 2.2.2 Multi-Agent RL for Slicing Resource Allocation

The paper [20] introduces a new resource allocation method in vehicular networks based on multi-agent reinforcement learning (MARL) for the efficient communication and computing resource allocation by network slicing. The authors consider the dynamic nature and heterogeneity of service demands in vehicular networks, presenting a technique that reduces system costs without compromising quality of service (QoS) for applications with different requirements.

#### 2.2.2.1 Methodology

The authors provide a solution to the resource allocation problem converted into a partially observable Markov decision process (POMDP), in which each base station (BS) is considered as an independent agent (regardless of being either macro base station (MBS) or small base station (SBS)). These agents collaborate for the resource allocation in terms of spectrum and computation, facilitating efficient operation of the network. To achieve this goal, we draw inspiration from the Multi-Agent Deep Deterministic Policy Gradient (MADDPG) algorithm, which performs offline training and real-time acting. The optimization problem seeks to minimize the total system cost of network slicing reconfiguration and takes into account QoS constraints and the dynamism of vehicular networks. Simulation results demonstrate that the proposed approach significantly outperforms classical and heuristic techniques in both cost reduction and QoS satisfaction, operating at an average power of 96.5%.

#### 2.2.2.2 Contributions

The authors proposed a complete allocation model for vehicular networks, which achieves resource allocation inside each reconfigurable network slice based on MARL.

A significant contribution is casting the slicing problem as a POMDP, where each base station is treated as an agent. This technique can deal efficiently with the dynamic and complex nature of vehicular networks, owing to its capability to make the resource allocation decision in real time. Results indicate that the MADDPG-based resource scheduling scheme effectively decreases users' cost and enhances QoS fulfillment compared to some baseline algorithms. Simulation results verify the effectiveness and scalability of the proposed scheme, which is expected to become an efficient technique for managing future vehicular networks.

#### 2.2.2.3 Limitations

Although the proposed MARL-based approach leads to significant enhancements, the work primarily addresses resource allocation at the network level and neglects some fine-grained issues, including task prioritization and per-vehicle requirements. As neural networks need to be trained, though the MADDPG algorithm works well, it highly depends on computation resources, which are hard to scale up under large size. Future studies may consider the joint application of more advanced RL techniques or dynamic resource allocation to heterogeneous vehicular services, making the system even more adaptable to actual environments.

### 2.2.3 Partial Task Offloading and Resource Allocation for VEC

The paper [21] introduces a joint partial task offloading and resource allocation for VEC in a dynamic environment. It works towards the joint optimization of task offloading and resource allocation in vehicular edge computing environments with computation-intensive vehicles experiencing dynamically changing network conditions.

#### 2.2.3.1 Methodology

The authors proposed a dynamic resource scheduling scheme based on task offloading and effective resource utilization to VEC systems. The problem is showed as an optimization problem minimizing a utility function in delay and energy. The goal is to achieve a trade-off between the energy and time consumed by tuning weight coefficients to reach optimal system performance. The decision variables for task offloading, channel allocation, and computation resource allocation are formulated. The objective is subject to various constraints such as maximum acceptable latency, limited resources, and energy consumption restrictions. We solve this problem by developing a multi-agent learning system with dynamic resource allocation and partial task offloading to enhance the system performance adaptively at any given time.

#### 2.2.3.2 Contributions

The main contribution of this paper is the design of a joint task offloading and resource allocation scheme that is able to work with the practical dynamics in vehicular scenarios. The optimization is achieved through the joint consideration of time delay and energy consumption, hence improving the system performance. The solution is embedded in a multi-agent learning framework, where each vehicle interacts with this system to make decentralized decisions. The authors are able to provide evidence that their approach improves upon other methods by having a lower latency and preserving the fairness in terms of resource assignment. Moreover, the method is demonstrated an effective adaptation to the dynamic network characteristic of IoV systems.

#### 2.2.3.3 Limitations

Despite its strengths, the work does not completely consider inter-vehicle mobility's influence on resource assignments. Also, the movements and connectivity can vary dynamically in vehicular networks which is not considered by the proposed mechanism. Although partial task offloading is studied in the paper, it does not take advantage of advanced task prioritization, which could allow the system to benefit from higher efficiency for high-priority tasks. Work may be done on the incorporation of mobility-aware mechanisms and more intelligent prioritization strategies to increase the scalability and adaptability of the system.

### 2.2.4 Asynchronous DRL for On-demand Resource Allocation

The paper [22] explores an innovative framework of collaborative task computing and on-demand resource allocation in VEC environments. It emphasizes V2V and V2I offloading proactive mechanism integration along with the distribution of computing resources over vehicles, edge servers, and cloud layers. In this work, they present an asynchronous DRL-based technique to solve the challenging problem of task offloading and resource allocation in dynamic vehicular networks.

#### 2.2.4.1 Methodology

Authors created a hybrid code that adopts both V2V and V2I assisted offloading cases. V2V offloading involves vehicles with computation capabilities assisting in processing tasks when they are idle, and V2I offloading leverages edge servers to help balance the loads across vehicles. The problem involving joint task offloading and resource scheduling is formulated as an optimization problem seeking to maximize the system utility while satisfying various service requirements of vehicular users. To address this challenge, the authors adopted an on-policy reinforcement learning algorithm, i.e.,

Asynchronous Advantage Actor-Critic (A3C), which is suitable for real-time operation and can manage the time-varying and high-dynamic nature of vehicular networks. It makes the algorithm take good consideration into the task and network.

#### 2.2.4.2 Contributions

This work contributes to VEC in many ways. Firstly, they propose a multi-resource framework that enables the joint resource allocation management for heterogeneous resources (e.g., vehicles, edge servers, and the cloud), enhancing resource utilization. Secondly, it designs a hybrid V2V and V2I offloading scheme that can provide more flexible and efficient task processing service. This design allows the system to more effectively serve the diverse user workloads by distributing the load across the edge and vehicle resources. Thirdly, a joint optimization of task computing and resource allocation is proposed in the paper based on asynchronous DRL to guarantee that decisions of optimal task offloading and scheduling are made. Finally, they did extensive simulation experiments to validate the effectiveness and efficiency of the proposed mechanism in terms of response latency and system utility, which performs better than conventional offloading approaches such as full offloading and random offloading.

#### 2.2.4.3 Limitations

Despite its merits, the paper does not tackle the scalability of the proposed system in large-scale vehicular networks, where it might be very challenging to include a large number of vehicles and edge servers. Moreover, although the paper used A3C to solve the problem of optimizing task offloading and resource scheduling and taking into account low-latency QoS requirements with real-time adaptation to network variation, they did not investigate advanced DRL techniques, which can also improve system adaptability toward network condition changes. They could have further explore utilizing Multi-Agent Reinforcement Learning (MARL) to model more complicated cases where there are multiple vehicles and edge nodes. Moreover, the paper does not investigate in detail how mobility affects the performance of the offloading strategy, which may be a room for improvement when they implement it into practical systems.

### 2.2.5 Dynamic Task Prioritization and Fair Resource Allocation

The paper [23] proposed a new dynamic task prioritization and fair resource allocation method for VEC systems. It emphasizes the need for coupling responsive task prioritization with considerations of vehicle vulnerabilities and fairness in resource distribution. The proposed approach is intended to accomplish the better performance of IoV networks by localizing different elements like mobility, heterogenous requirements for tasks and real-time network situations.

#### 2.2.5.1 Methodology

The authors developed a dynamic task scheduling scheme which dynamically adjusts to the instantaneous status of network and vehicle mobility so as to facilitate efficient resource allocation, while at the same time prioritizing safety-critical tasks. The urgency calculation is influenced by both the dynamic task urgency and elapsed time and applies to task prioritization. This framework also includes a fairness metric as the Weighted Jain's Fairness Index (WJFI) and the Normalized Cumulative Deviation Index (NCDI) to keep a high level of balanced allocation across various task urgencies. We then cast the above problem as an Mixed Integer Nonlinear Programming (MINLP) problem where they want to maximize the utility of system while maintaining fairness in the distribution of resources. The model also incorporates vehicle vulnerability terms to allocate more resources to the most vulnerable vehicles.

#### 2.2.5.2 Contributions

This paper makes a significant contribution to resource allocation in IoV networks through an innovative dynamic task prioritization scheme that is capable of adapting to the network states and the vehicle movement. As one of the aspects to address, the authors have characterized the vehicle attack surface in the resource allocation model and made this possible for a fair allocation (i.e., no task starvation to less critical tasks). Further, the authors suggest a global fairness model that integrates WJFI and NCDI to achieve fair resource allocation among various categories of tasks. The effectiveness of the proposed framework is demonstrated in numerical results, and it has been shown that outperforms other models both on task executing success rate and fairness for varying network situations.

#### 2.2.5.3 Limitations

It is evident that the developed model is effective but does not sufficiently tackle the scalability issue when dealing with large-scale IoV networks where the number of vehicles and tasks can indeed be high. Additionally, on the other hand, although in this paper the authors have static task priorities, they do not explore more advanced methods such as reinforcement learning for optimizing allocation decisions at greater depth and complexity in dynamic environments. Future work may seeks to improve the scalability and flexibility of the model, especially in dense vehicular networks, as well as the incorporation of advanced learning algorithms to enable the model to make real-time decisions.

### 2.2.6 LLM-Based Task Offloading and Resource Allocation

The paper [24] deals with the problems of task offloading and resource allocation in Digital Twin (DT) systems structured by Edge Computing networks, specifically for vehicular networks. This paper articulates a new model leveraging the complementary properties of DTs and LLMs for optimized task offloading and resource allocation in VEC-enabled systems.

#### 2.2.6.1 Methodology

The authors propose an LLM-based decentralized task offloading framework in which tasks created by vehicles are executed either at the vehicle itself or mobilized to RSUs, which are in the imminence of a vehicle. The LLM model is deployed for intelligent decision-making and takes into consideration multiple attributes such as task type, vehicle speed, network status, and the distance between vehicles to RSU. The approach profiles tasks and vehicles before employing LLM-based prompts to rank tasks. These scores drive the decision of what to offload, taking into account computational capability and timeliness for time-critical tasks. The paper also discusses the incorporation of DT models for simulating the behavior of vehicles and its application in better resource allocation decisions.

#### 2.2.6.2 Contribution

The contributions of the paper are threefold. First, it introduces a novel hybrid paradigm that utilizes LLMs to execute and offload tasks within VEC networks that guarantee vehicles take the most advantageous decisions in executing tasks. Second, we present a decentralized resource allocation scheme to increase the scalability and flexibility of the system in dynamic environments. Thirdly, the present paper further conducts a comprehensive performance evaluation to show that the proposed approach can greatly reduce latency and energy cost over traditional approaches. In conclusion, this study shows the promise of leveraging DTs and LLMs for real-time decision support in IoV applications.

#### 2.2.6.3 Limitation

Although the proposed approach appears to work well, several issues are recognized in this paper. One limitation is the reliance on accurate vehicle and task profiling, which is not always possible in practice where very limited data is available. Moreover, vehicular environments are dynamic, and we should take dynamic decisions based on the context for VEC as their dynamism can bring randomness to the task offloading scenario, which can make an impact on the system behavior when it is confronted with

a highly varying environment. The authors further mention that the scalability of the system could be limited in large-scale IoV networks consisting of many vehicles and RSUs. Lastly, they have only utilized the reasoning capabilities of LLMs in a limited manner, whereas we are focusing on using them even more.

## 2.3 Comparison Among State-of-the-Art Works

Table 2.1 shows a comparative analysis of the existing task offloading and resource allocation approaches in VEC. This table also shows what the works include and what they lack in comparison to our proposed solution.

From the table 2.1, we can see that task prioritization and context consideration are only handled by the existing work [23]. All the works do latency optimization, whereas work [23] misses out on this. Only the work [24] take account of integrating LLMs in their framework, although it is limited.

Table 2.1: Comparison of State-of-the-Art VEC Approaches

| System | Task Prioritization | Context Consideration | Latency Optimization | Cost Optimization | LLM Integration | Algorithm Nature |
|---|---|---|---|---|---|---|
| **MAPTD3 [19]** | X | X | ✓ | ✓ | X | DRL-based |
| **MADDPG [20]** | X | X | ✓ | ✓ | X | MARL |
| **Partial Offloading [21]** | X | X | ✓ | ✓ | X | Multi-agent Learning |
| **A3C-based VEC [22]** | X | X | ✓ | ✓ | X | DRL (A3C) |
| **Dynamic Prioritization [23]** | ✓ | ✓ | ✓ | X | X | Optimization-based |
| **LLM-Based Task Offloading [24]** | X | ✓ | ✓ | ✓ | ✓ | LLM & DT |

## 2.4 Summary

In this chapter, we reviewed the key state-of-the-art literature on task offloading in VEC. We also reviewed task prioritization and cost and latency optimization in the existing works. We also looked with special emphasis on context-aware prioritization as well as the incorporation of LLMs. Although many work has already been done to optimize the task execution and network efficiency, problems of dynamic decision making, real-time task scheduling based on priority, and network diversity still exist. Our proposed LEVEC framework is presented in the next section which provides a solution to these challenges. It efficiently makes task prioritization decisions based on dynamic situations and latency-cost optimization with the help of LLM.

# 3. Proposed Methodology of LEVEC

## 3.1 Introduction

This chapter is solely focused on describing our proposed system approach and design. This section is divided into four major sections, and each of them dives into different parts of our proposed system. Section 3.1 is an introduction to this chapter, which is followed by section 3.2, where the system framework is described. In section 3.3 we have shown how we have formulated the problem and proposed methodologies for trade-offs between minimizing the cost and delay of tasks, which is very important to the system as a whole. In section 3.4 we have described our proposed LLM-based solution for taking optimal decisions. Lastly, section 5 contains the conclusion of the chapter.

## 3.2 Proposed Framework and Assumption

This section gives a full overview of our proposed LEVEC framework. As illustrated in Fig. 3.1, the framework consists of three key elements, including the vehicle layer, the RSU layer, and the task scheduling & offloading mechanism. In this structure, vehicles create high-performance tasks that cannot be entirely executed locally because of the constraints of onboard resources. In order to overcome this limitation, the vehicles offload their tasks to RSUs in close proximity with abundant resources for executing the tasks. The RSUs, meanwhile, leverage LLMs to prioritize tasks according to their urgency with their contextual metadata. After the task prioritization, tasks are scheduled in an M/M/1 queue, in which decisions of local processing or task offloading to RSUs are dynamically taken depending on the available resources and deadlines to complete each task. The execution of the whole task or part of the task can be processed locally by the vehicle or offloaded to its close-by RSU depending on computational capacity and latency.

As shown in Fig. 3.1, where each of the vehicles sends their task metadata to the closest RSU. The RSU layer, which is equipped with the Llama-3 (8B) model, processes this metadata and prioritizes all tasks based on a task priority prompt. The scheduling queue is an M/M/1 queue of the tasks sorted by priorities. Whether the task is

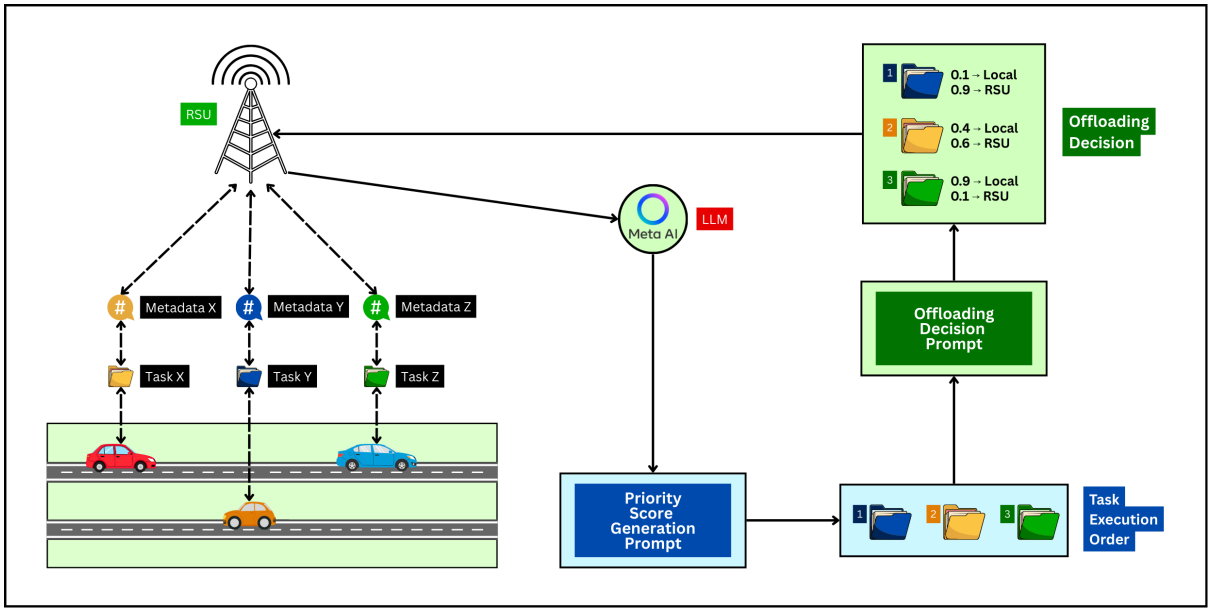



Figure 3.1: Task Offloading System Framework in VEC Environment

completed locally or offloaded to an RSU nearby to further process depends on the remaining deadline and available vehicle resources for each task. Then the task to be offloaded will go to a nearby RSU, and the task can be dynamically divided into partial, i.e., partially offloading, or full, i.e., fully offloading, based on the size of the task and how time-critical each task is.

Moreover, the task offloading location and percentage decision are determined by another prompt, which selectively adjusts the size of the task to be offloaded based on the available computational resources of vehicles and the urgency of tasks and priority score calculated by LLM. This mechanism allows tasks to be implemented effectively, and high-priority tasks are guaranteed to meet the deadline, in addition to reducing the system cost in general.

In the model we presented, we make the following assumptions so as to simplify system model and make it usable in practice. First, we consider a vehicular environment containing multiple vehicles and RSUs deployed throughout the urban scenario. These RSUs have enough computation capacity as edge servers and can carry out the heavy computations that vehicles cannot do because of their limitations in terms of onboard processing.

We consider computationally intensive and time-sensitive tasks to be mostly generated by each vehicle, e.g., those related to safety-critical applications, infotainment, and navigation. All of these tasks come with certain metadata: the task type, data size, time deadline, and vehicle-specific parameters such as speed and network conditions. The metadata will be transmitted by vehicles to the nearest RSU that reasons about the priority for the task according to their associated context, using Llama-3 (8B).

To prioritize a task, we consider that the metadata processing capabilities in the LLMs of RSUs are capable to analyze metadata and running a priority score prompt for each task. This score was then fed into the M/M/1 queue in order to sort the tasks in which they would be executed. The task offloading is based on another prompt mechanism that assumes tasks are processed partially/fully by an RSU depending on the available computational resources; such decisions follows from context-based priority.

Furthermore, we assume that each task is offloadable to one RSU at most, which simplifies the distribution of tasks and reduces the complexity of resource scheduling. The offloaded tasks are fully or partially executed by the RSU depending on the size of the task and available computing resources at the RSU.

## 3.3 Formulation of Proposed System

In this section, we have shown the mathematical formulation of the proposed LEVEC framework. A set of vehicles and RSUs comprises the system. Each vehicle creates some computationally demanding tasks that it cannot accomplish due to limited computational resources onboard. To solve this problem, the tasks of vehicles are assigned to nearby RSUs with enough computing capability. LLM is deployed on RSUs to score the priority of each task according to its contextual metadata, such as the time sensibility of the tasks, vehicle speed, network status and resources.

The framework have a mechanism for making intelligent decisions on the offloading tasks, taking into account into task priority, available computational resources, network states, and task deadlines. Each task may be performed locally on the vehicle or offloaded partially/fully to an adjacent RSU, according to the computation capability of the vehicle and necessity of each task.

To mathematically model the considered system, we first define the sets of vehicles $\mathcal{U}$ and RSUs $\mathcal{R}$ as well as the parameters that characterize tasks and resources. Task offloading decision is formulated as a binary decision problem, in which tasks must be locally executed (flag $\alpha_{u,k} = 1$) or be offloaded to RSU (flag $\alpha_{u,k} = 0$). If a task gets partially offloaded, the offloading fraction of the partial task to be received by a RSU is denoted as $\tilde{\alpha}_{u,k} \in [0, 1]$.

The connectivity constraint makes certain the task can be offloaded to an RSU only if the vehicle is within its coverage, modeled by $\chi_{u,r}$, which is a binary indicator. Additionally, the optimization problem is a multi-objective minimization that trades off task delay and cost over all vehicles and RSUs. The priority value generated by the LLM is incorporated in the optimization framework for task offloading decisions, according to the sense of urgency of tasks.

The system under consideration is formulated as an MINLP optimization problem in which the decision of whether or not to offload and the allocation of resources are binary and continuous variables, respectively. The problem comes with constraints, like the RSU capability constraint, the bandwidth assignment constraint and the task deadline requirement. In addition, performance of the task processing at RSUs is analyzed using queuing M/M/1 model and the rate of arrival and service are modeled to maximize system utility depending on different network conditions.

In the subsequent sub-sections, we present the mathematical abstraction of the task offloading scenario and its objectives in terms of decision variables, constraints, and optimization problems. Furthermore, we develop an LLM-based priority score to govern the placement and scheduling of tasks for obtaining the optimal execution strategy with minimal system constraints.

### 3.3.1 Initial Considerations

We consider a vehicular edge computing (VEC) system composed of a set of vehicles $\mathcal{U} = \{1, \ldots, U\}$ and a set of roadside units (RSUs) $\mathcal{R} = \{1, \ldots, R\}$. The system operates in a slotted time manner, and all system dynamics are analyzed in each time slot $t$. For simplicity of presentation, the time index is omitted whenever it does not lead to ambiguity.

Each vehicle $u \in \mathcal{U}$ generates a set of computation tasks indexed by $k \in \{1, \ldots, K_u\}$. A task $(u, k)$ is characterized by the tuple $\left\{B_{u,k}, O_{u,k}, C_{u,k}, \tau_{u,k}^{\max}\right\}$, where $B_{u,k}$ denotes the input data size, $O_{u,k}$ denotes the output data size, $C_{u,k}$ represents the required number of CPU cycles, and $\tau_{u,k}^{\max}$ is the maximum tolerable execution delay. The onboard CPU frequency of vehicle $u$ is denoted by $f_u$, while $f_r^{(u,k)}$ denotes the CPU clock speed allocated to task $(u, k)$ at RSU $r$. The total computational capacity of RSU $r$ is $F_r^{\max}$. The uplink transmission rate from vehicle $u$ to RSU $r$ is denoted by $r_{u,r}$, which is obtained based on Shannon's capacity formula. The total bandwidth of RSU $r$ is $W_r$.

### 3.3.2 Decision Variables

We introduce the binary variable $\alpha_{u,k} \in \{0, 1\}$ to indicate the execution mode of task $(u, k)$. Specifically, $\alpha_{u,k} = 1$ implies that the task is fully executed locally at the vehicle, whereas $\alpha_{u,k} = 0$ indicates that the task involves remote execution, either partially or fully. The variable $\tilde{\alpha}_{u,k} \in [0, 1]$ denotes the portion of the task that is executed locally at the vehicle, and $X_{u,k,r} \in \{0, 1\}$ is a binary offloading indicator that takes the value 1 if task $(u, k)$ is offloaded to RSU $r$. For each task, at most one RSU can be selected for offloading. The variable $f_r^{(u,k)} \geq 0$ denotes the computational resource allocated to task $(u, k)$ at RSU $r$, and $\phi_{u,r} \in [0, 1]$ denotes the fraction of bandwidth allocated by RSU $r$ to vehicle $u$.

Table 3.1: Notation

| Symbol | Meaning |
|---|---|
| $u$ | vehicle index ($u \in \mathcal{U}$) |
| $k$ | task index of vehicle $u$ ($k = 1, \ldots, K_u$) |
| $r$ | RSU index ($r \in \mathcal{R}$) |
| $B_{u,k}$ | input data size (bits) of task $(u, k)$ |
| $O_{u,k}$ | output/result data size (bits) of task $(u, k)$ |
| $C_{u,k}$ | required CPU cycles (cycles) of task $(u, k)$ (estimated by profiling) |
| $\tau_{u,k}^{\max}$ | maximum tolerated delay of task $(u, k)$ |
| $f_u$ | CPU Clock speed of vehicle $u$ (onboard) (cycles/s) |
| $f_r^{(u,k)}$ | Clock speed of the allocated cpu to task $(u, k)$ at RSU (cycles/s) $r$ |
| $F_r^{\max}$ | total CPU capacity (cycles/s) of RSU $r$ |
| $r_{u,r}$ | uplink transmission rate from vehicle $u$ to RSU $r$ (bits/s) (from Shannon) |
| $W_r$ | total bandwidth (Hz) available at RSU $r$ |
| $\alpha_{u,k} \in \{0, 1\}$ | 1 if task $(u, k)$ executed locally; 0 if candidate for offload |
| $\tilde{\alpha}_{u,k} \in [0, 1]$ | portion scalar for attribution (local weight) |
| $X_{u,k,r} \in \{0, 1\}$ | 1 if task $(u, k)$ is offloaded to RSU $r$ (at most one $r$) |
| $p_{u,k} \in [0, 1]$ | priority score for task $(u, k)$ provided by LLM (exogenous) |
| $\chi_{u,r}$ | connectivity indicator: 1 if vehicle $u$ is in RSU $r$'s coverage, else 0 |
| $(x_u, y_u)$ | 2D position of vehicle $u$ (time index omitted) |
| $(x_r, y_r)$ | 2D position of RSU $r$ |
| $R_r$ | coverage radius of RSU $r$ |
| $d_{u,r}$ | euclidean distance between vehicle $u$ and RSU $r$ |
| $h_{u,r}$ | channel power gain (including pathloss and small-scale fading) |
| $N_0$ | noise power spectral density (W/Hz) |
| $P_u^{\text{tx}}$ | vehicle $u$ transmit power (W) |
| $\eta_{\text{pa}}$ | power amplifier efficiency ($0 < \eta \leq 1$) |
| $\kappa_u, \kappa_r$ | CPU energy coefficients at vehicle / RSU |
| $\lambda_r, \mu_r$ | arrival rate and service rate (tasks/s) at RSU $r$ (for M/M/1) |

When a task is fully executed locally, i.e., $\tilde{\alpha}_{u,k} = 1$, no RSU is selected for offloading and hence $\sum_r X_{u,k,r} = 0$. If the task is fully offloaded, i.e., $\tilde{\alpha}_{u,k} = 0$, exactly one RSU is selected such that $\sum_r X_{u,k,r} = 1$. Similarly, in the case of partial offloading where $0 < \tilde{\alpha}_{u,k} < 1$, exactly one RSU is selected to process the offloaded portion of the task, which again leads to $\sum_r X_{u,k,r} = 1$.

The offloading decision must satisfy the RSU coverage constraint, which is given by

$$X_{u,k,r} \leq \chi_{u,r}, \quad \forall u, k, r, \tag{3.1}$$

where $\chi_{u,r}$ is a connectivity indicator that equals 1 if vehicle $u$ lies within the coverage area of RSU $r$, and 0 otherwise.

### 3.3.3 Task Profiling Model

Each vehicle $u \in \mathcal{U}$ generates a set of computation tasks indexed by $k = 1, \ldots, K_u$. In order to enable LLM-assisted decision-making, we construct two structured datasets that describe the vehicle context and the task characteristics, respectively. These datasets are then fused to form a unified semantic profile, which is provided as input to the large language model (LLM) for task priority estimation.

#### 3.3.3.1 Vehicle Feature Set

The vehicle feature dataset characterizes the mobility state, communication condition, and computing capability of each vehicle. For vehicle $u$, the feature vector is defined as

$$\text{VehicleFeat}_u = \left\{ (x_u, y_u),\ v_u,\ f_u,\ P_u^{\text{tx}},\ \mathcal{R}_u,\ D_u^{\text{RSU}},\ \mathcal{C}_u \right\}, \tag{3.2}$$

where $(x_u, y_u)$ denotes the current geographic coordinates of the vehicle, $v_u$ is its velocity, and $f_u$ represents the onboard CPU frequency that reflects the local computing capability. The term $P_u^{\text{tx}}$ denotes the uplink transmission power, while $\mathcal{R}_u$ represents the set of RSUs that are reachable by vehicle $u$. The distance between the vehicle and its associated RSU is denoted by $D_u^{\text{RSU}}$. Furthermore, $\mathcal{C}_u$ captures contextual information such as network condition or link quality, which is mapped to a normalized numeric scale in the interval $[0, 1]$.

#### 3.3.3.2 Task Feature Set

The task feature dataset captures the computational, temporal, and semantic attributes of each task. For task $(u, k)$, the feature vector is given by

$$\text{TaskFeat}_{u,k} = \left\{ \text{Id}_{u,k},\ \Theta_{u,k},\ B_{u,k},\ O_{u,k},\ C_{u,k},\ \tau_{u,k}^{\max},\ \alpha_{u,k}^{\text{rule}},\ \text{Ctx}_{u,k} \right\}, \tag{3.3}$$

where $\text{Id}_{u,k}$ is the unique identifier of the task and $\Theta_{u,k}$ denotes the task type such as *Infotainment*, *Safety*, or *Navigation*. The parameters $B_{u,k}$ and $O_{u,k}$ indicate the input and

output data sizes, while $C_{u,k}$ denotes the required number of CPU cycles. The maximum tolerable delay is represented by $\tau_{u,k}^{\max}$. The coefficient $\alpha_{u,k}^{\text{rule}}$ is obtained from classical scheduling policies such as Priority Scheduling, Round Robin, FCFS, or SJF, depending on the data characteristics. Finally, $\text{Ctx}_{u,k}$ denotes semantic or environmental contextual information related to the task, including urgency and execution purpose.

#### 3.3.3.3 Final Combined Profile

The two feature sets are merged into a single semantic profile that is provided as input to the LLM:

$$\text{Profile}_{u,k} = \Big\{\text{VehicleFeat}_u,\ \text{TaskFeat}_{u,k}\Big\}. \tag{3.4}$$

#### 3.3.3.4 LLM-Based Priority Estimation

Given the combined profile $\text{Profile}_{u,k}$, the LLM outputs a normalized priority score $p_{u,k} \in [0,1]$ as

$$p_{u,k} = \mathcal{F}_{\text{LLM}}\big(\text{Profile}_{u,k}\big), \tag{3.5}$$

where $\mathcal{F}_{\text{LLM}}(\cdot)$ denotes the mapping function implemented by the large language model. The resulting priority score reflects both the computational importance of the task and the current operational state of the vehicle, and is subsequently integrated into the optimization framework to guide offloading and resource allocation decisions.

### 3.3.4 Vehicle Position and RSU Coverage Model

In the context of the vehicular edge computing environment, the positions of all vehicles are assumed to be known at each time instant $t$. Specifically, the location of vehicle $u$ at time $t$ is denoted by the coordinates $(x_u(t), y_u(t))$. Similarly, each roadside unit (RSU) $r$ is deployed at a fixed geographical location $(x_r, y_r)$ and is characterized by a circular coverage area with a radius of $R_r$.

The distance between a vehicle $u$ and an RSU $r$ at time $t$ is determined using the Euclidean distance formula, given by:

$$d_{u,r}(t) = \sqrt{(x_u(t) - x_r)^2 + (y_u(t) - y_r)^2}. \tag{3.6}$$

To model the connectivity between vehicles and RSUs, we introduce a binary indicator variable $\chi_{u,r}(t)$. This indicator is defined such that $\chi_{u,r}(t) = 1$ if the vehicle $u$ is within the coverage radius of RSU $r$, i.e., if $d_{u,r}(t) \leq R_r$. Conversely, $\chi_{u,r}(t)$ equals 0 if the vehicle is outside the coverage area.

As a result, for any given task generated by vehicle $u$, offloading to RSU $r$ is only possible if $\chi_{u,r}(t) = 1$. This connectivity constraint is mathematically expressed as:

$$X_{u,k,r} \leq \chi_{u,r}, \quad \forall u, k, r. \tag{3.7}$$

This ensures that the task offloading decision respects the real-time connectivity conditions imposed by vehicle mobility and RSU coverage.

### 3.3.5 Uplink Transmission Rate Based on Shannon Capacity

When vehicle $u$ communicates with RSU $r$, it is allocated a certain bandwidth $W_{u,r}$ in Hertz. The achievable uplink transmission rate is derived from Shannon's capacity formula, which is expressed as:

$$r_{u,r} = W_{u,r} \log_2 \left(1 + \mathrm{SNR}_{u,r}\right) \tag{3.8}$$

$$\mathrm{SNR}_{u,r} = \frac{P_u^{\mathrm{tx}} h_{u,r}}{N_0 W_{u,r}}, \tag{3.9}$$

where $P_u^{\mathrm{tx}}$ denotes the transmit power of vehicle $u$, $h_{u,r}$ is the channel gain (encompassing path loss and fading effects), and $N_0$ is the noise power spectral density.

The connectivity constraint in Equation (3.7) ensures that the transmission occurs only when the vehicle is within the RSU's coverage area, thereby enabling a feasible communication link.

### 3.3.6 Delay Modeling

The total delay associated with task execution is composed of several components, including local execution delay, transmission delay, and RSU processing delay.

#### 3.3.6.1 Local Execution Delay

When a portion of the task is executed locally on the vehicle, the local execution delay is given by:

$$T_{u,k}^{\mathrm{loc}} = \frac{\tilde{\alpha}_{u,k} C_{u,k}}{f_u}, \tag{3.10}$$

where $\tilde{\alpha}_{u,k}$ represents the fraction of the task processed locally, $C_{u,k}$ is the number of required CPU cycles, and $f_u$ is the vehicle's onboard CPU frequency.

#### 3.3.6.2 Transmission Delay

For the portion of the task that is offloaded, the uplink transmission delay to RSU $r$ is expressed as:

$$T_{u,k,r}^{\mathrm{tx}} = \frac{(1 - \tilde{\alpha}_{u,k}) B_{u,k}}{r_{u,r}}, \tag{3.11}$$

where $B_{u,k}$ is the input data size of the task and $r_{u,r}$ is the uplink transmission rate derived from Shannon's formula.

#### 3.3.6.3 RSU Processing Delay

Once the task data arrives at the RSU, the RSU processing delay is determined by the computational capacity allocated to the task. This delay is given by:

$$T_{u,k,r}^{\text{proc}} = \frac{(1-\tilde{\alpha}_{u,k})B_{u,k} \times C_{u,k}}{f_r^{(u,k)}}, \tag{3.12}$$

where $f_r^{(u,k)}$ represents the computational capacity of RSU $r$ allocated to task $(u,k)$.

#### 3.3.6.4 Queueing Delay Using M/M/1 Model

To capture the queuing effects at the RSU, we model the task queue as an M/M/1 system. The average queuing delay at RSU $r$ is given by:

$$T_{u,k,r}^{\text{queue}} = \frac{\rho_r}{\mu_r(1-\rho_r)}, \quad \rho_r = \frac{\lambda_r}{\mu_r} < 1, \tag{3.13}$$

where $\lambda_r$ is the task arrival rate and $\mu_r$ is the service rate, which is approximated by $\mu_r \approx f_r / \mathbb{E}[C_{u,k}]$.

#### 3.3.6.5 Priority-Adjusted Queueing Delay

To incorporate task priority, the baseline queueing delay is adjusted by a priority score $p_{u,k} \in [0,1]$. The effective queueing delay for task $(u,k)$ at RSU $r$ is defined as:

$$T_{u,k,r}^{\text{eff\_queue}} = \frac{T_r^{\text{queue}}}{1+\eta_Q p_{u,k}}, \tag{3.14}$$

where $\eta_Q$ is a tunable parameter that controls the sensitivity of the delay adjustment. This priority-aware queueing delay is then used to refine the computation reliability, which is expressed as:

$$\mathcal{R}_{u,k,r}^{\text{comp}} \approx 1 - \exp\Big(-\mu_r\big(\tau_{u,k}^{\max} - T_{u,k,r}^{\text{eff\_queue}}\big)\Big). \tag{3.15}$$

#### 3.3.6.6 Propagation and Transition Delay

The propagation delay, including access setup and handshaking, is modeled as:

$$T_{u,k,r}^{\text{prop}} = (1-\tilde{\alpha}_{u,k})\hat{T}_{u,r}^{\text{prop}} X_{u,k,r}, \tag{3.16}$$

where $\hat{T}_{u,r}^{\text{prop}}$ is the baseline propagation delay between vehicle $u$ and RSU $r$.

#### 3.3.6.7 Total Task Delay

The overall delay for task $(u,k)$ is the sum of the local execution delay and the cumulative delays incurred from offloading, including propagation, transmission, queueing, and RSU processing. Therefore, the total delay is expressed as:

$$T_{u,k} = T_{u,k}^{\text{loc}} + \sum_r X_{u,k,r}\big(T_{u,k,r}^{\text{prop}} + T_{u,k,r}^{\text{tx}} + T_{u,k,r}^{\text{eff\_queue}} + T_{u,k,r}^{\text{proc}}\big). \tag{3.17}$$

### 3.3.7 Cost Modeling

This subsection presents the economic model used to quantify the monetary cost associated with both computation and communication in the proposed vehicular edge computing framework. The total cost incurred by each task is composed of three main parts, namely the local computing cost at the vehicle, the remote computing cost at the RSU, and the bandwidth usage cost during uplink transmission.

#### 3.3.7.1 Computing Cost Model

The computing cost is modeled as a linear function of the number of CPU cycles consumed, weighted by the corresponding per-cycle unit price. When a portion of task $(u, k)$ is executed locally at the vehicle, the incurred local computing cost is expressed as

$$\mathrm{C}_{u,k}^{\mathrm{cpu,loc}} = c_u^{\mathrm{cpu}} \Big( \tilde{\alpha}_{u,k}\, C_{u,k} \Big), \tag{3.18}$$

where $c_u^{\mathrm{cpu}}$ denotes the unit price of CPU cycles at the vehicle and $\tilde{\alpha}_{u,k}$ represents the fraction of the task processed locally.

When the remaining portion of the task is offloaded and executed at RSU $r$, the corresponding remote computing cost is given by

$$\mathrm{C}_{u,k,r}^{\mathrm{cpu,rsu}} = c_r^{\mathrm{cpu}} \Big( (1 - \tilde{\alpha}_{u,k})\, C_{u,k} \Big) X_{u,k,r}, \tag{3.19}$$

where $c_r^{\mathrm{cpu}}$ is the per-cycle price charged by RSU $r$, and the binary variable $X_{u,k,r}$ ensures that the cost is incurred only when the task is actually offloaded to RSU $r$.

#### 3.3.7.2 Bandwidth Cost Model

To capture the communication expenditure, we adopt a per-(Hz·s) pricing scheme, in which the operator charges proportionally to the product of allocated bandwidth and transmission duration. Under partial offloading, only the remote fraction $(1 - \tilde{\alpha}_{u,k})$ of the task input data contributes to the uplink transmission cost. Accordingly, the bandwidth cost incurred at RSU $r$ for task $(u, k)$ is expressed as

$$\begin{aligned}
\mathrm{C}_{u,k,r}^{\mathrm{bw}} &= c_r^{\mathrm{Hzs}} \big( W_{u,r} \cdot T_{u,k,r}^{\mathrm{tx}} \big) X_{u,k,r} \\
&= c_r^{\mathrm{Hzs}} \left( W_{u,r} \cdot \frac{(1 - \tilde{\alpha}_{u,k}) B_{u,k}}{W_{u,r} \log_2(1 + \mathrm{SNR}_{u,r})} \right) X_{u,k,r} \\
&= c_r^{\mathrm{Hzs}} \left( \frac{(1 - \tilde{\alpha}_{u,k}) B_{u,k}}{\log_2(1 + \mathrm{SNR}_{u,r})} \right) X_{u,k,r},
\end{aligned} \tag{3.20}$$

where $c_r^{\mathrm{Hzs}}$ denotes the unit price per (Hz·s). It is evident from this formulation that when the task is fully processed locally, i.e., $\tilde{\alpha}_{u,k} = 1$, the bandwidth cost becomes zero.

#### 3.3.7.3 Total Cost Formulation

By aggregating the local computing cost, the remote computing cost, and the bandwidth usage cost, the overall cost incurred by task $(u,k)$ is obtained as

$$\mathrm{C}_{u,k} = \mathrm{C}_{u,k}^{\mathrm{cpu,loc}} + \sum_{r\in\mathcal{R}} X_{u,k,r}\left(\mathrm{C}_{u,k,r}^{\mathrm{cpu,rsu}} + \mathrm{C}_{u,k,r}^{\mathrm{bw}}\right). \tag{3.21}$$

This total cost expression provides a unified metric for evaluating the economic efficiency of task execution strategies under different offloading and resource allocation decisions.

### 3.3.8 Optimization Framework

We assume that the system-level trade-off parameters $\omega_D$ and $\omega_C$ are provided externally by the LLM or by the system operator, and satisfy the normalization condition

$$\omega_D + \omega_C = 1, \qquad \omega_D, \omega_C \geq 0. \tag{3.22}$$

To ensure that delay and cost terms are comparable in scale, both metrics are normalized prior to optimization. For each task $(u,k)$, the normalized delay and normalized cost are defined as

$$\bar{T}_{u,k} = \frac{T_{u,k}}{T^{\max}}, \qquad \bar{C}_{u,k} = \frac{C_{u,k}}{C^{\max}}, \tag{3.23}$$

where $T^{\max}$ denotes the maximum tolerable delay and $C^{\max}$ represents the maximum acceptable execution cost.

Each task is associated with a priority value $p_{u,k} \in [0,1]$, produced by the LLM based on semantic and contextual features. A higher priority score indicates greater importance of the task and hence amplifies its contribution in the objective function.

The joint delay-cost minimization problem is formulated as

$$\mathbf{P1}: \quad \min_{\alpha_{u,k},\tilde{\alpha}_{u,k},X_{u,k,r}} \sum_{u\in\mathcal{U}} \sum_{k=1}^{K_u} p_{u,k}\left(\omega_D\,\bar{T}_{u,k} + \omega_C\,\bar{C}_{u,k}\right) \tag{3.24}$$

**Subject To**

$$C_1 : T_{u,k} \leq \tau_{u,k}^{\max}, \quad \forall u \in \mathcal{U},\ k = 1, \ldots, K_u, \tag{3.25}$$

$$C_2 : \sum_{r \in \mathcal{R}} X_{u,k,r} = 1 - \alpha_{u,k}, \quad \forall u \in \mathcal{U},\ k = 1, \ldots, K_u, \tag{3.26}$$

$$C_3 : X_{u,k,r} \leq \chi_{u,r}, \quad \forall u \in \mathcal{U},\ k = 1, \ldots, K_u,\ r \in \mathcal{R}, \tag{3.27}$$

$$C_4 : \alpha_{u,k} \in \{0,1\}, \quad X_{u,k,r} \in \{0,1\}, \quad \forall u \in \mathcal{U},\ k = 1, \ldots, K_u,\ r \in \mathcal{R}, \tag{3.28}$$

$$C_5 : \rho_r = \frac{\lambda_r \, \mathbb{E}[C_{u,k}]}{f_r} < 1, \quad \forall r \in \mathcal{R}, \tag{3.29}$$

$$C_6 : \tilde{\alpha}_{u,k} = 1 \Rightarrow \sum_{r \in \mathcal{R}} X_{u,k,r} = 0, \quad \forall u \in \mathcal{U},\ k = 1, \ldots, K_u, \tag{3.30}$$

$$C_7 : \tilde{\alpha}_{u,k} = 0 \Rightarrow \sum_{r \in \mathcal{R}} X_{u,k,r} = 1, \quad \forall u \in \mathcal{U},\ k = 1, \ldots, K_u, \tag{3.31}$$

$$C_8 : 0 < \tilde{\alpha}_{u,k} < 1 \Rightarrow \sum_{r \in \mathcal{R}} X_{u,k,r} = 1, \quad \forall u \in \mathcal{U},\ k = 1, \ldots, K_u. \tag{3.32}$$

Here, Constraints (3.25)–(3.32) define the feasible region for the offloading and resource allocation problem in vehicular edge computing. Constraint (3.25) ensures that the total delay experienced by each task $(u, k)$, including local execution, transmission, RSU processing, and queueing, does not exceed its maximum tolerable latency $\tau_{u,k}^{\max}$. Constraint (3.26) guarantees consistency between the local execution fraction and RSU offloading variables, ensuring that each task is either partially or fully offloaded without violating execution mode rules. Constraint (3.27) enforces connectivity limitations, allowing task offloading only to RSUs within the coverage of the vehicle, as indicated by the connectivity indicator $\chi_{u,r}$. Constraint (3.28) defines the domain of decision variables, specifying that both the local execution flags $\alpha_{u,k}$ and RSU selection indicators $X_{u,k,r}$ are binary. Constraint (3.29) guarantees queue stability at each RSU by keeping the traffic load below its service capacity. Finally, Constraints (3.30)–(3.32) specify the execution modes for tasks: fully local tasks do not use any RSU resources, fully remote tasks must be offloaded to exactly one RSU, and partially offloaded tasks are divided between local execution and exactly one RSU, ensuring proper enforcement of offloading rules.

## 3.4 Proposed LLM Enhanced Solution

### 3.4.1 Task Priority Scoring Mechanism

It is important to mention that in LEVEC, the task priority scoring mechanism decides how tasks based on their urgency and resources are treated. To calculate the priority score of each task, a combination of structure metadata, network conditions, its closeness to an RSU, and physical realism are used. The resulting score between 0.000 and 1.000 assists in the subsequent task scheduling and offloading decisions.

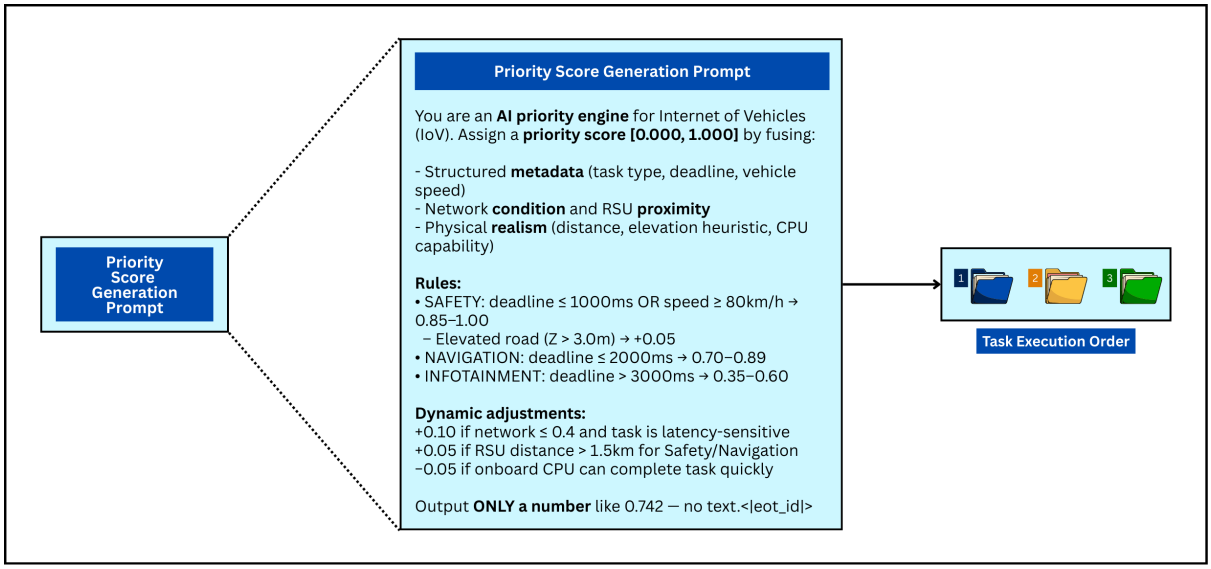



Figure 3.2: Workflow of LEVEC's priority score generation and task execution ordering

As shown in Figure 3.2, the mechanism is designed to adjust the priority score of individual tasks dynamically. This depends on some key factors, which are:

- **Structured Metadata:** This is composed of type of task, deadline, and speed of vehicle. Safety, navigation, and infotainment-related tasks are differently addressed according to the urgency and the criticality of execution.
- **Network Condition and RSU proximity:** Tasks with low network values (or tasks that are far from an RSU) receive priority modification in order to guarantee the execution time is in the acceptable bound.
- **Physical Reality:** The distance of the vehicle to the RSU, the height of the task location, and the CPU computational power on the vehicle are considered in prioritization.

The priority score for each task is calculated using the following rules:

- **Safety:** If there is a safety-related task, this is the most important. A priority score is assigned between 0.85 and 1.00 for tasks with less than equal 1000 ms deadline or vehicle velocity greater than equal 80 km/h. If the vehicle is on a bridge or overpass ($Z \geq 3.0m$) then the score is increased by +0.05.
- **Navigation:** Navigation tasks with a deadline inferior to 2000 ms are assigned priority levels between 0.70 and 0.89.
- **Infotainment:** Infotainment-related tasks with deadlines superior to 3000 ms get priority scores in the range of 0.35 and 0.60.

Furthermore, dynamic adjustments are made based on network conditions and task sensitivity:

- +0.10 if the network condition $\leq 0.4$ and the task is latency-critical.
- An additional +0.05 is added if the RSU range greater to 1.5 km in safety or navigation applications.
- -0.05 if the on-board CPU can execute the task quickly, reducing off-loading.

The final priority score is a sum of all these factors that helps decide whether the job will be executed locally or offloaded to an RSU. The priority mechanism allows the system to prioritize safety-critical and important tasks, which enables the best utilization of task performance and resources.

### 3.4.2 Task Weight Factor Determination

In VEC, the task offloading and resource allocation should consider both **latency** and **cost**. The dynamic task prioritization and scheduling policy of the LEVEC framework calculates a **priority score** according to system-dependent parameters that are both delay-sensitive and cost-sensitive. Here we outline the weight factor determination of task prioritization, emphasizing the main biases and inference model.

#### 3.4.2.1 Delay-Sensitive System Bias

The delay-sensitive system bias ensures that actions directed to tight deadlines and to resources on which to perform critical safety tasks, have a higher priority. The priority score depends on the job type, the due date, vehicle speed, and other context. Those with tighter schedules, such as safety-critical tasks (e.g., collision avoidance), are to be given higher priority to avoid execution delay. The rules that the system applies for support of delay-based task prioritization are:

- **Safety**: Jobs with deadlines lesser than $100ms$ are not assigned priority despite having priority scores between 0.85 and 1.00. If the deadline is in $100 - 250ms$, the weight score is between [0.70, 0.85].
- **Navigation**: Scores for tasks with deadlines lesser than 300 ms are ranged between 0.60 and 0.75. Tasks with timeouts $300ms$ are given scores in the range of 0.40 to 0.55.
- **Infotainment**: These are always given a priority score 0.40, and with deadlines $500ms$, a score between 0.10 and 0.30 is chosen.

In addition, certain conditions elicit an adaptation:

- **Short deadlines (6 km):** +0.05 to +0.10 for the network delays.
- **High speed ($90km/h$):** +0.05 if a very tight penalty exists, since it is more urgent.
- **Large data size ($4000KB$) with tight time limits ($100ms$):** 0.05 for the possibility of missing the deadline.

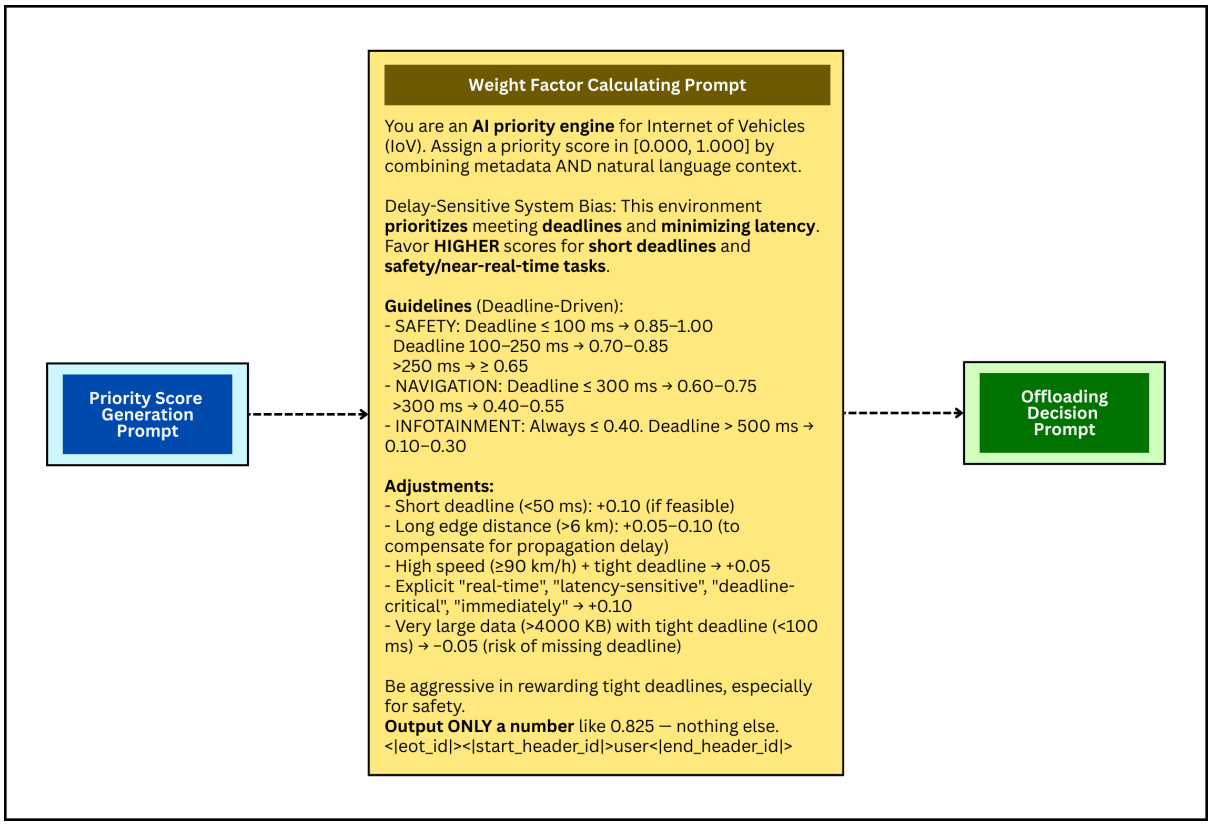



Figure 3.3: LEVEC's weight factor generation mechanism for dynamic contexts

#### 3.4.2.2 Cost-Sensitive System Bias

The **cost-sensitive system bias** is for low-cost nature, where many of these tasks are not critical and their objective becomes to reduce **energy consumption** and **bandwidth usage**. The system is designed to trade off minimizing energy consumption with task responsiveness. The less important jobs, for example, **infotainment** and **non-critical navigation**, are marked down in priority level to inhibit insolent taking of resources. The following cost directives are taken into account by our system:

- **Safety:** Important tasks might be preferred over low-resource ones, especially at high speed ($\geq 80km/h$).

- **Navigation:** These tasks are cost-sensitive when the urgency is not very high.

  **Infotainment:** Non-critical Infotainment tasks are assigned the least priority score to save resources.

Present task score modifications due to resource usage:

- **High data size (greater than 3000 KB):** Non-safety-critical tasks are penalized with a 0.1 score.
- *Good Network Conditions* ($\geq 0.85$): Low Urgency tasks can be slightly penalized Cheap Offloading (-0.05).
- **Long RSU distance (**$6km$**):** Safety-critical tasks are calibrated with +0.05 of actuation due to extra network latency.

#### 3.4.2.3 LLM-based Inference Model

LEVEC utilizes the **Llama 3-8B** model for inference towards task prioritization. The system utilizes information such as task metadata (task type, deadline and data size) and natural language context (e.g., urgency signals like "emergency," "immediate," "critical") to calculate priority score from 0.000 to 1.000. The LLM makes context-aware decisions based on both numeric data and the context of work to schedule tasks intelligently.

The deduction is reflected in the following steps:

- The system aggregates task information, which may include metadata and environmental input.
- The LLM calculates a priority score according to the context in which the task is operating, as described by task prioritization rules for delay- and cost-sensitive systems.
- This score is employed to make the offloading decision. Tasks with large scores will be offloaded to RSUs or executed locally depending on the available resources.

#### 3.4.2.4 Global Weight Calculation

Having given all tasks priority scores, LEVEC calculates two global weights that are delay weight and cost weight. These weights are obtained after averaging the priority score among all tasks and help to focus on reducing latency versus cost. The system is able to refine its task scheduling strategy using such weights and accordingly reallocate tasks.

The global weights allow the system to gain and lose **focus on latency** as a task-solving modus, considering the **cost-effectiveness** in low-load scenarios.

The combination of the task prioritization affected by delay sensitivity and decisions based on the cost makes the solution scalable and efficient for real-time vehicular edge computing.

### 3.4.3 Task Offloading Decision Mechanism

The task offloading decision mechanism in LEVEC aims to autonomously determine how much of a vehicular task should be offloaded to an RSU. As shown in Fig. 3.4, this mechanism defines the fraction, $\delta$, of the task to be offloaded to the RSU, where:

- $\delta = 0.0$ indicates that the task is executed entirely locally on the vehicle.
- $\delta = 1.0$ signifies that the task is completely offloaded to the RSU.
- $\delta = 0.7$ implies that 70% of the task is offloaded to the RSU, with 30% remaining for local execution on the vehicle.

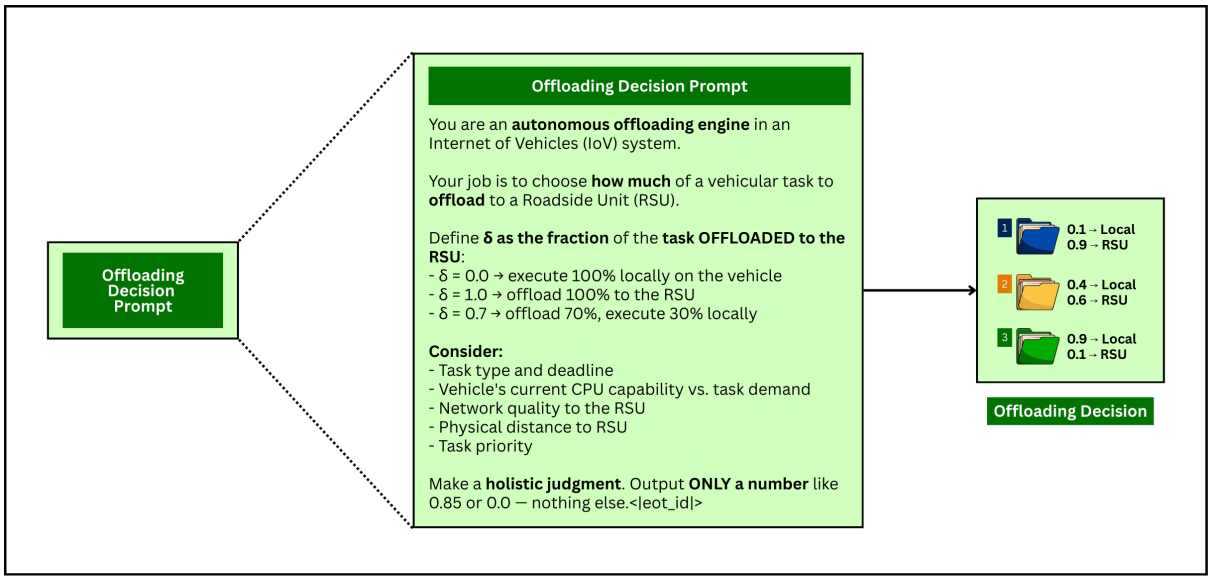



Figure 3.4: LEVEC's task offloading calculation and location determination framework

We have considered these factors to decide on the offloading decision:

- **Task Type and Deadline:** The importance of the task and its deadline has a significant impact on the amount of offloading to be performed.
- **Vehicle's CPU Capacity Vs. Task Demand:** If the vehicle's onboard CPU can process the task well, it might prefer local execution. However, when the onboard computation and/or storage resources are inadequate, offloading is employed.

- **Network Quality to RSU:** Network status is assessed here also. If the vehicle is distant from the RSU or there is bad network condition, it may affect the offloading decision for executing as soon as possible.
- **Physical Distance to RSU:** The vehicle's physical distance to the closest RSU is assumed, where the closer RSUs are prioritized to offload data.
- **Task Priority Score:** The LLM-generated priority score, as described in the previous subsection, makes it ready to serve in higher priority. Higher priority tasks can be less offloaded to ensure low latencies.

The offloading process is simulated via a natural language model prompt that provides a value to $\delta$ given the above parameters. The system determines the offloading fraction by considerin task metadata, network conditions, and the computational capability of the vehicle.

In our implementation, it has been implemented via a command prompt where the priority score of the task is checked and the offloading for that particular task is decided. When the model receives a task request, it processes its metadata (i.e., type, deadline, vehicle's current CPU capacity, network condition, and the physical distance to RSU) for every operating task. Given these inputs, the amount of task to be offloaded is decided, and the value of $\delta$ is output.

We assume that a prompt for deciding whether to offload follows the rule set, which considers task urgency, vehicle computation power, and network status. The decision is then in the form of a single number between 0.0 and 1.0, where if = 1.0 implies an absolute offloading scenario, while it corresponds to full execution locally. The existence of $\delta$ is a reflection of how the system makes a holistic judgement about task offloading with all the constraints for meeting deadlines and minimizing overall system cost.

The decision-making benefits from the fact that the LLM is aware of and can take into account task metadata along with vehicle context, network quality, and physical state to perform efficient task offloading. A high-level explanation of such a decision mechanism is available in the code provided, and its result (i.e., the decision) can be exploited for optimization in task execution planning.

## 3.5 Task Metadata Generation and Utilization

In this section, for our proposed system, we introduce its task metadata generation, which is crucial to derive the task priority and offloading of IoV based on vehicles. The metadata is encoded with several key properties of tasks, including task-specific attributes (e.g., type, size and deadline), vehicle-related factors (e.g., speed, location

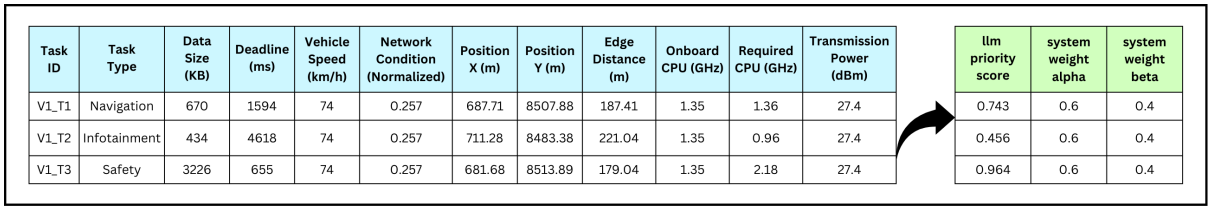
and on- board CPU frequency) as well as network conditions (e.g., signal strength and distances from RSU).

The metadata is created on the fly for each vehicle and task. The set of vehicle-specific characteristics is enhanced with task-dependent (e.g., jitter in speed and position, network values, and deadline) characteristics. These variations are recorded as metadata information, which is preserved for later use. Every task is given a unique identifier and attached to the vehicle it belongs to, such that there exists a direct relation between the vehicle's available computational resources, its task load, and the underlying network as well as the context situation.

### 3.5.1 Task Metadata Structure

The task metadata is organized into a structured dataset that includes the following features:

- **Task ID**: Unique identifier for each task, which corresponds to the vehicles in sequential order or the task.
- **Task Type**: The type of the task, such as *Safety*, *Navigation*, or *Infotainment*. Each task type has different requirements for urgency and resource allocation.
- **Data Size (KB)**: The size of the task data in kilobytes, which influences the bandwidth and computational requirements for processing the task.
- **Deadline (ms)**: The maximum tolerable delay for the task, critical for determining how the task is prioritized and whether it can be processed locally or offloaded.
- **Vehicle Speed (km/h)**: The current speed of the vehicle, which impacts the urgency of time-sensitive tasks like safety or navigation.
- **Network Condition (Normalized)**: A numeric value representing the quality of the network, with values ranging from 0 (poor) to 1 (excellent), affecting the feasibility of offloading tasks to nearby RSUs.
- **Position (X, Y, Z)**: The geographic coordinates of the vehicle. The position is crucial for determining the proximity to RSUs, which is essential for task offloading.
- **Onboard CPU Frequency (GHz)**: The processing capability of the vehicle's onboard unit, which determines if the vehicle can handle the task locally or needs to offload it to a nearby RSU.
- **Edge Node Distance (km)**: The distance from the vehicle to the nearest RSU, which impacts the potential delay during task offloading and transmission.



| Task ID | Task Type | Data Size (KB) | Deadline (ms) | Vehicle Speed (km/h) | Network Condition (Normalized) | Position X (m) | Position Y (m) | Edge Distance (m) | Onboard CPU (GHz) | Required CPU (GHz) | Transmission Power (dBm) | llm priority score | system weight alpha | system weight beta |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| V1_T1 | Navigation | 670 | 1594 | 74 | 0.257 | 687.71 | 8507.88 | 187.41 | 1.35 | 1.36 | 27.4 | 0.743 | 0.6 | 0.4 |
| V1_T2 | Infotainment | 434 | 4618 | 74 | 0.257 | 711.28 | 8483.38 | 221.04 | 1.35 | 0.96 | 27.4 | 0.456 | 0.6 | 0.4 |
| V1_T3 | Safety | 3226 | 655 | 74 | 0.257 | 681.68 | 8513.89 | 179.04 | 1.35 | 2.18 | 27.4 | 0.964 | 0.6 | 0.4 |

Figure 3.5: Task Metadata

- **Required CPU (GHz)**: The computational power required to complete the task, compared against the vehicle's onboard CPU capability to decide on task offloading.
- **Transmission Power (dBm)**: The power used for uplink communication, affecting the network conditions and bandwidth requirements for offloading tasks.
- **Context**: A brief textual description providing additional information about the task, such as whether it is safety-critical, navigation-related, or infotainment, to help the system understand the task's urgency and nature.

### 3.5.2 Task Metadata Utilization in Offloading Decisions

The task metadata has great importance on both task prioritization and offloading decisions. As soon as the metadata is generated, it is given to the LLMs, which then reason with it to generate a priority score and calculate how much of the task should be offloaded and how much it should be executed locally. The LLM uses contextual information such as task urgency, network conditions, and vehicle resource constraints to assign an appropriate offloading fraction. The task can be executed entirely locally on the vehicle or partially/fully on RSU by offloading, depending on the available resources and the priority score.

We can make sure that the important tasks are given more priority and often tried for instant execution and the less important ones are executed with flexibility of time. By adjusting this in a dynamic manner, we are making sure the system optimizes the usage of available resources to an optimal level and timely execution of each task according to context.

## 3.6 Engineering and Societal Considerations

Engineering creativity results in breakthroughs of technology. However, such advances must conform to moral, environmental, and social guidelines. It is very much necessary for engineering solutions to not just improve technical efficiency but also consider the implications of technology for people, the planet, and even our profession. This

section describes societal, environmental, and ethical considerations of our proposed approach towards VEC that we propose, highlighting the necessity to take into account responsibility at each stage in engineering.

### 3.6.1 Societal Impacts of Engineering Solutions

The interaction between engineered systems and nature also translates into engineering system and social interactions, which means that there are perceivable payoffs as well as problems associated with technological advances. Our proposed scheme involving task offloading and LLM-supported task prioritization system has the potential to enhance task processing and resource management in vehicular networks. But we also need to think about the societal effects of these inventions.

On the plus side, such an approach can result in large efficiency improvements in urban transportation. We are, therefore, making the vehicle on-board unit more efficient and powerful to perform some complex computation tasks if we allow vehicles to offload computing to RSUs. It could increase accessibility where resources are limited and improve safety where real-time monitoring is necessary, such as in autonomous driving systems.

But there are risks as well. Autonomous transportation could eliminate jobs, especially in the classic driving profession. Moreover, processing sensitive information (i.e., vehicle location and task metadata) has privacy implications. Guaranteeing the security of data and privacy of users is an important issue to be considered, and we need to also take into account the problem caused by the sharing of transmitted data between vehicles and RSUs.

Through an ambitious embrace of human-centered design, our goal is to maximize positive impacts such as increased urban mobility and safety while minimizing unintended consequences. The proposed solution guarantees that social welfare is maximized while ensuring fairness, privacy, and equity.

### 3.6.2 Environment and Sustainability Considerations

In a period when preserving the environment is becoming concerning day after day, engineering solutions should be made to actively contribute to carbon saving and encourage green behaviors. We present a novel model for the VEC system, which also focuses on low-power task offloading and computational resource management to reduce their environmental impact.

The low power consumption is one of the most important aspects of our design. By shifting computation-intensive tasks to RSUs, which have better computational power, the load of on-board computations in vehicles is decreased. This contributes to the overall energy consumption of the system by reducing the energy used for one vehicle.

In addition, the VEC concept is also consistent with a number of United Nations Sustainable Development Goals (SDGs), such as SDG 9 (Industry, Innovation, and Infrastructure) and SDG 11 (Sustainable Cities and Communities). We help smart, thriving cities become even more successful by managing and modernizing their transportation systems to better meet the needs of a rapidly changing society.

The designs have also been developed to reduce resources and waste. As an example, our system reduces non-vital task execution and optimally distributes tasks according to the vehicle resource availability. We also make use of green data practices for optimizing offloading so that the environmental penalties of both transmitting data and processing a task is minimized.

### 3.6.3 Ethical and Professional Responsibility

Ethical and professional responsibility is a foundational issue in all engineering practices. Our system is governed by specific ethical guidelines written into the IEEE Code of Ethics, particularly centered on public safety, transparency, and fairness. We also adhere to the guidelines in the ACM Code of Ethics, ensuring that our design and implementation respect privacy, secure data, and protect individuals' rights.

The privacy of data is one of the major concerns in our system, as it deals with collecting and transmitting sensitive information associated with both users and vehicles. We make sure that all possible data exchange between vehicles and RSUs will be done in a responsible way and with the driver's consent (if required). The system is developed under a secure data policy to avoid access and misuse of the personal information.

Apart from privacy concerns, its LLM-based prioritization of tasks is also an ethical minefield. We also guarantee that the used AI models are transparent and that their decisions can be explained for accountability in task offloading and execution. Responsible AI use, especially for safety-critical applications such as autonomous cars, is crucial to avoid harm and ensure that AI systems operate within ethical boundaries.

We follow ethical guidelines, ensure privacy and consent throughout our work, and promise that responsible AI is at the core of this system from development to deployment. This guarantees that the technology will be put to use for the good of society in a transparent and responsible way.

## 3.7 Conclusion

This chapter introduces our LLM-enhanced task offloading and execution framework for ensuring context-aware task prioritization and scheduling while optimizing the latency and cost of tasks by leveraging the reasoning power of LLMs in VEC. This

framework optimizes task offloading and execution decisions for the vehicles via LLMs too.

By combining the reasoning expertise of LLMs in VEC, we can make sure decisions are made totally based on context, which leads to better optimization of delay and cost management in the system as a whole. Doing this allows the system to be implemented on a large scale to build up smart cities and traffic infrastructure.

# 4. Results and Discussion

## 4.1 Introduction

In this section, a simulation has been conducted of the proposed LEVEC system and performed a comparative analysis with two other research works, which are based on LLM-DT [24] and MAPTD3 [19]. The evaluation of all these systems was performed on Python 3.1 on the Windows operating system.

The LLM-DT framework uses LLMs with DT, but it has some limitations compared to LEVEC. While LEVEC utilizes LLMs more in system optimization, LLM-DT depends more on MARL. It provides an additional challenge for the LLM-DT framework as it works with many complex mechanisms at a same time, such as LLMs, DTs and MARL whereas LEVEC focuses solely on LLMs, which reduces system complexity and overall performance.

The MAPTD3 approach, on the other hand, does latency and cost optimization with a DRL-based mechanism. It doesn't take account in task prioritization or context consideration. Here, our LEVEC has an advantage of using context-based task prioritization and LLM integration. This is a big difference maker in the long run, as the system is more optimized and well-performing under LEVEC.

Our framework is implemented by Python, and the simulation results are generated with the OpenAI GYM custom environment in Microsoft Visual Studio. The setup for simulation is described in Section 4.2. The evaluation metrics are described in Section 4.3. A detailed discussion about the experimental results in terms of comparison is given in Section 4.4. The last section of this chapter, Section 4.5, is devoted to conclusion and summary.

## 4.2 Simulation Environment

The simulation environment of LEVEC is modeled using the OpenAI GYM custom environment in Microsoft Visual Studio Code.

The simulation scenario, as summarized in Table 4.1, is used to verify the performance of the proposed LEVEC framework. In that scenario the system is composed of 200 vehicles, which generate tasks with different data sizes from 200 KB to 5000 KB.

Table 4.1: Simulation Parameters

| **Parameter Descriptions** | **Assigned Values** |
|---|---|
| Number of Vehicles | Up to 200 |
| Number of RSUs | Up to 10 |
| Data Size | 200 to 5000 KB |
| Deadline | 10 to 2000 ms |
| Vehicle Speed | 10 to 80 km/h |
| Network Condition | 0.0 to 1.0 |
| Edge Distance | 20 to 500 m |
| Onboard CPU | 1.2 to 3.2 GHz |
| Required CPU | 0.5 to 3.0 GHz |
| Transmission Power | 20 to 35 dBm |
| LLM Priority Score | 0.0 to 1.0 |
| System Weight $\omega_D$ | 0.0 to 1.0 |
| System Weight $\omega_C$ | 0.0 to 1.0 |
| Offload Decision | 0.0 to 1.0 |

These operations are characterized by the time constraints they impose, ranging from 10 milliseconds to 2000 milliseconds, which covers testing for system responsiveness and compliance with latency constraints. The vehicles are assumed to have an onboard computation power between 1.2 GHz and 3.2 GHz, while the tasks may require processing capacity from 0.5 GHz up to 3.0 GHz Which constitutes a set of computational demands inside the vehicular network area under study, are simulated this way.

There are some key parameters that affect the task offloading decision in the system. The number of RSUs is from 1 to 10, and each RSU provides computing resources for processing the task offloading. Vehicles move at a variable speed, varying from 10 km/h to 80 km/h, thus tasking priorities and decisions are set on-line. It mixes with another important factor, the network condition normalized between 0 and 1 to emulate different levels of congestion and reliability based on which offloading decisions made by agents are affected as well as execution times.

The edge distance (i.e., vehicle-RSU distances) is from 20 meters to 500 meters, and it influences both the communication delay and decision-making in these V2I scenarios. The transmission power varies between 20 and 35 dBm, while LLM priority scores (0.0 to 1.0) are essential for the urgency of each task. The factors are further utilized to make the optimal offloading decisions that vehicles could offload tasks partially or fully unto RSUs in accordance with the weight of system $\omega_D$ and $\omega_C$ (both from 0.0 to 1.0). These weight factors enable easy trade-off between minimizing cost and minimizing latency.

The system is intended to dynamically make offloading decisions, taking all these parameters into account. The offload decision parameter is set to change between 0.0 and 1.0, since when the parameter takes a value of 0.0, there is no offloading (all processing occurs locally on board the vehicle), while when the parameter is assigned a value of 1.0, there is full offloading to the RSU, and if the value is in-between them, then partial offloading occurs. This scenario will allow us to have a full assessment of the framework for optimizing both latency and cost efficiency of task offloading, based on real-time conditions in the vehicular network.

## 4.3 Performance Metrics

In our comparative analysis, we employ some important performance metrics to compare and contrast the efficiency and effectiveness of task offloading systems under investigation. Such indicators are crucial to feedback for the entire system performance in terms of economic affordability and responsiveness. Before that, we have done evaluation of two state-of-the-art LLM models to determine which one is the best for LEVEC.

- **LLM Evaluation:** Two LLM models, which are Llama 3 - 8B and Mistral 7B are evaluated based on their reasoning expertise. Tasks are given to them, and their expertise on priority score assignment, weight factor assignment, and offloading decision based on context is evaluated to select the best LLM for the overall system.

- **System Cost:** This metric reflects the overall resource cost for task offloading, including energy consumption, computation workload, and network use. Lower system cost, therefore, also indicates better optimization for resource management and operation. It is an important parameter to be considered in the assessment of the technical and economic viability of the offloading strategy, particularly for broad-scale deployments, which require a reduction in operational costs.

- **System Latency:** It refers to the time difference (i.e., delay) between when a task is called and when it completes. This directly reflects the response time of the system, especially in real-time applications such as safety and navigation. Latency reduction is essential for meeting the deadlines of applications and hence achieving optimal performance in a dynamic vehicular edge computing world. A smaller latency equals a more responsive system where tasks can be performed time-based effectively.

Through rigorous analysis of these performance metrics, we can assess the trade-off relationship between cost and efficiency, which could offer precious verifications

regarding the pros and cons of different task offloading schemes. These measures act as the benchmarks for designing our proposed framework to optimize the cost and improve the latency efficiently considering vehicular edge computing.

## 4.4 Simulation Results

### 4.4.1 LLM Evaluation

#### 4.4.1.1 Average task priority score

This comparison checks the differential priority scores of Mistral 7B and Llama 3–8B with respect to three different types of vehicular tasks: Infotainment, Navigation, and Safety. As can be seen from Fig. 4.1, the two models indeed yield a strong hierarchical ordering according to functional criticality but with quantitatively different emphasis profiles. Llama 3–8B achieves higher priority scores in all task categories; the difference becomes particularly pronounced for Safety tasks: Llama gets a score of 0.932, while Mistral only attains 0.916. Likewise, for Navigation tasks, Llama assigns 0.810 compared to Mistral's 0.787 and for infotainment, the least important category, both are closely matched at around 0.521.

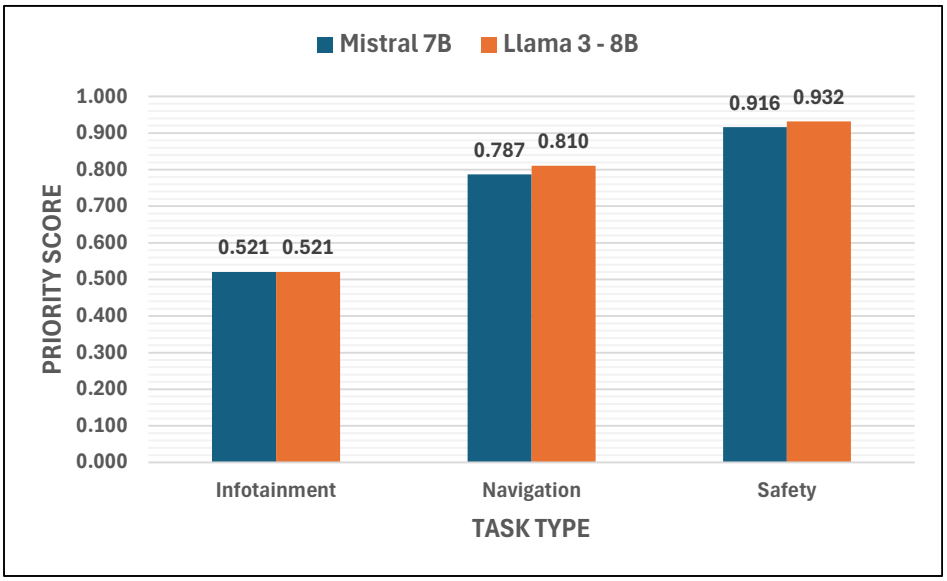



Figure 4.1: Average task priority score by different task types

We can say that the discrepancy in scores is a result of differences in each model's internal weighing of task importance. Llama 3–8B exhibits a less risk-averse attitude towards scheduling, especially when it comes to safety-critical activities, showcasing a design philosophy that has placed a high premium on functional assurance and

operational reliability. Mistral 7B, despite the same ordinal hierarchy (safety first, navigation second, and infotainment last), has a stricter scale in its scoring, which can be interpreted as a more balanced security resource predicament. These findings suggest that Llama 3–8B may be more well-suited for tasks that demand unambiguous, high-stakes prioritization of behavior, while Mistral 7B might suit computing systems where resource sharing and gradational priority are encouraged.

#### 4.4.1.2 Average offloading decision

The offloading decision patterns of both Mistral 7B and Llama 3–8B exhibit different types of strategic bias towards task offloading. This can be seen in Figure 4.2 where both models show task-type specific offloading preferences, but Llama 3–8B always prefers better offloading ratios for all tasks types based on context, especially for Navigation and Safety tasks. For Safety-critical type of tasks, Llama achieves an offloading decision score which is significantly better than Mistral, which demonstrates a more intense preference to the offload for safety-related processing in order to attain reliability and real-time validation via lower response time based on distributed computing. Likewise, for Navigation Llama scores better against Mistral, indicating a method focused on always giving guiding decisions by servicing through auxiliary computational assistance.

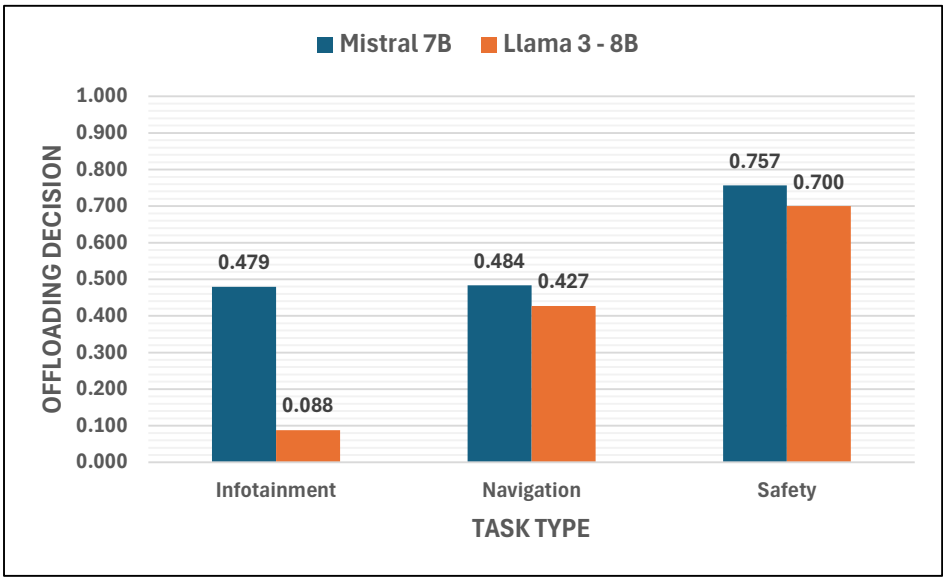



Figure 4.2: Average offloading decision by different task types

On the contrary, Infotainment operations get quite low offloading scores for both models, agreeing that non-safety-critical tasks should be computed locally to save bandwidth as well as latency. Nevertheless, Mistral achieves a higher score in this

aspect, suggesting a less balanced resource distribution, while Llama takes a more polarized approach whereby it offloads crucial functions to the cloud but keeps non-critical ones on-device. This pattern indicates that Llama 3–8B is a context- and risk-aware offloading policy, making more intensive use of RSUs computation power for higher operational expense functions. This would seem to be an excellent match for Llama given its design and particularly relevant in the context of autonomous and safety-critical systems where computational reliability can only ever be established through redundancy and distributed verification.

#### 4.4.1.3 System weight factor values

In this part, we compare the effectiveness of these two LLMs in determining adaptive system weights factors in dynamic scenarios. In the figure 4.3, the weights $\omega_D$ (latency-sensitive priority) and $\omega_C$ (cost-sensitive priority) specify dynamic computational resource allocation in response to different mix of tasks. We also see that Llama 3-8B always tends to issue larger $\omega_D$ in the latency intensive case, with a maximum of 0.735 under the 70/30 latency/cost distribution as opposed to Mistral7B's value of 6.15. In contrast, we find that in more cost-aware scenarios Llama 3-8B is significantly more economical with $\omega_C$ of 0.715 at a 30/70 distribution which is higher than the efficiency of Mistral 7B's $\omega_C$ value of 0.645. This twofold superior behaviour manifests that Llama 3-8B has a stronger and more situation-aware resource prioritization mechanism, which responds effectively to workload changes when it aims at both performance and efficiency.

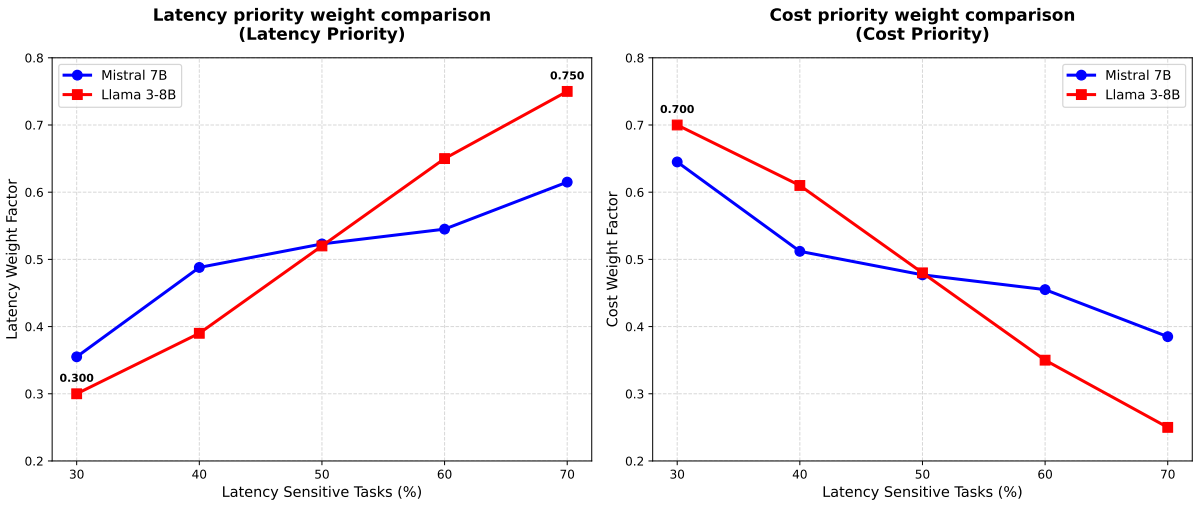



Figure 4.3: System weight factor assignment comparison among both LLM models

The advantage for the Llama 3-8B is further contrasted here in the difference in adaptability between both designs. Although both models become converged to an

extreme when the workloads are perfectly balanced (e.g., 50/50), Llama 3-8B shows a broad enough dynamic range of weight adjustment as $\omega_D$ is varied from 0.285 and to 0.735, and $\omega_C$ from 0.265 through 0.715, indicating that priority parameters move by as much as 158%. By comparison, Mistral 7B has a narrower tuning range of percentage change in $\omega_D$ (73%: 0.355–0.615) and $\omega_C$ (67%: 0.385–0.645). A wider adaptive range of the Llama 3-8B makes it more aggressive at customizing resource partition to workload traits, significantly boosting system responsiveness under a low latency budget yet reducing cost-effectiveness for limited resources. As a result, Llama 3-8B is the better model for deployments, with strong and task-aware resource management being preferred over specialization at more than ample performance in mixed operational environments.

Based on the figures 4.1, 4.2 and 4.3, we can say that Llama 3-8B is a better fit for LEVEC as it takes more context-aware decisions, which would benefit the system in the long run.

### 4.4.2 Impacts of Varying Number of Vehicles

In this subsection 4.4.2, we will see how varying the number of vehicles affects average latency and average cost of the system. We will compare by varying the number of vehicles. We will set the vehicle to 4, 6, 8, and 10. Here we have set 2 RSUs for the setup. Then we have compared the average latency and average cost of LEVEC with MAPTD3 and LLM-DT.

#### 4.4.2.1 Average latency comparison

With the figure 4.4 we can compare the average latency among LEVEC, MAPTD3, and LLM-DT under different vehicle densities, which reveals significant differences in terms of scalability, adaptability, and efficiency in vehicular edge computing. With more vehicles (4 to 10), all three approaches experience a rise in latency (as anticipated due to higher task arrivals, e2e resource sharing with the edge, and more queuing delay), though it's not a large change. Nevertheless, the rate of growth of latency is very different across methods, and it demonstrates the efficiency or lack thereof in their decisions to perform tasks.

We can see LEVEC under all the vehicle densities has low latency, presenting nice scalability and stability with the heavy load. Meanwhile, at lower vehicle numbers, the latency gain is not that much. It proves that LEVEC makes very good use of edge and local resources by making offloading decisions intelligently. As the vehicle count increases, LEVEC exhibits a slowly increasing delay with orderly progression, which indicates that it helps in congestion avoidance and supports opportunistic scheduling. Such behavior is mainly due to the incorporation of context-aware reasoning

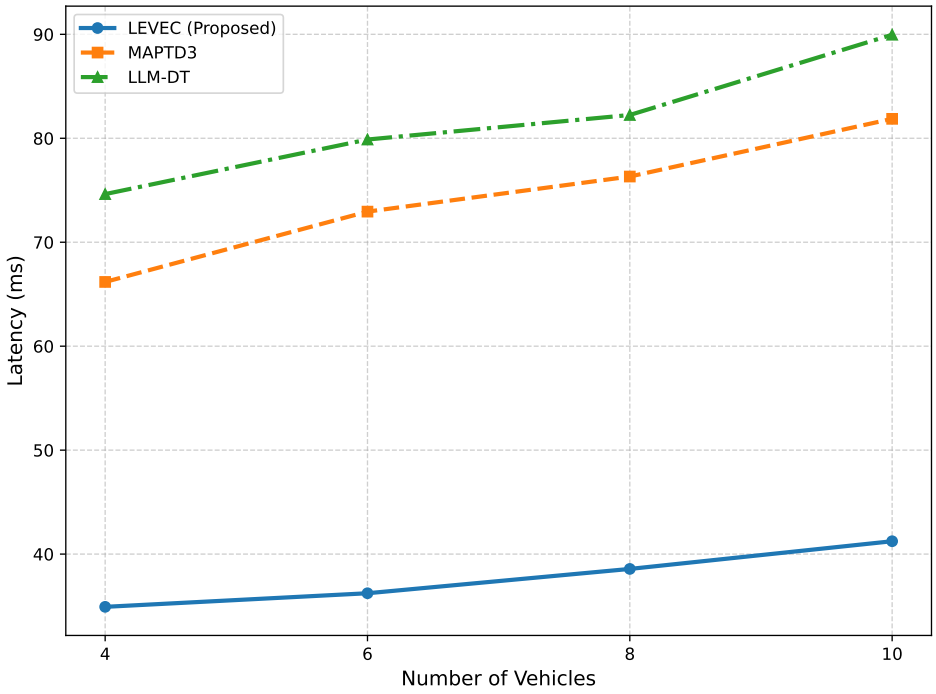



Figure 4.4: Average latency comparison on varying vehicle density

and adaptive execution techniques, which enable LEVEC to dynamically make task scheduling decisions with degrees of importance and minimize both offloading and queuing overhead. Its continuous nature of the LEVEC curve suggests its capacity to accommodate increasing load without abrupt performance drops.

On the other hand we can see that MAPTD3 exhibits significantly higher latency compared to LEVEC at all vehicle counts, and its increasing rate of latency is faster when system load becomes heavier. Although MAPTD3 is empowered by reinforcement learning based resource management and works for dynamic scenarios, All of these approaches essentially suffer the shortage that the decision making process mainly depends on numeric state display and reward-oriented optimization. As the traffic load becomes denser, MAPTD3 suffers from increased end-to-end delay mainly due to slow convergence in highly dynamic scenarios and ignorance of task semantics or urgency. The increasing latency gap between MAPTD3 and LEVEC with increase in number of vehicles, emphasizes the drawbacks of completely learning based methods in presence of heterogeneity as well as with time-dependent vehicle workloads.

LLM-DT method has the greatest latency in the three methods and it also increases fastest with the number of vehicles. While the use of Digital Twin based modeling and contextual reasoning in LLM-DT incurs a higher processing latency by adding overhead due to state synchronization, model inference, and abstract layer mechanisms. As more vehicles edge into the overheads, they accrue, leading to an instant increase in latency that is sharper than LEVEC and MAPTD3. The curve shows that LLM-DT

delivers increased system awareness. However, it is not efficient for heavy-loaded networks and therefore not appropriate for delay-sensitive applications under dense vehicular environments.

In the comparative evaluation of latency performance under varying vehicular densities, our proposed LEVEC framework demonstrates a substantial and consistent superiority over both the MAPTD3 and LLM-DT approaches. Across the tested range of 4 to 10 vehicles, LEVEC achieves an average reduction in latency of approximately 50.9% relative to MAPTD3 and 54.7% relative to LLM-DT. This marked improvement underscores the efficacy of LEVEC's design in mitigating computational and communication delays, particularly as system scale increases. Notably, while both MAPTD3 and LLM-DT exhibit pronounced non-linear latency escalation with rising vehicle counts, LEVEC maintains a comparatively stable and low-latency profile, thereby highlighting its robustness and scalability in dynamic edge computing environments. These results affirm the proposed architecture's potential as a viable solution for latency-sensitive vehicular network applications. This makes LAVEC the most suitable for latency optimization.

#### 4.4.2.2 Average cost comparison

Average cost comparisons for LEVEC (proposed LLM), MAPTD3, and LLM-DT with different vehicle densities illustrate the different performance in terms of cost-effectiveness and resource usage. The average cost of all 3 methods will keep increasing as the number of vehicles goes up from 4 to 10, which is in a natural increase trend because computational resources and communication are consumed more. The cost reflects how well their offloading and execution strategies work.

LEVEC delivers the lowest average cost in all vehicle counts, which shows better cost-aware decision-making and efficient utilization of edge and local resources. Under lower vehicle densities, LEVEC minimizes operational cost by preferring local or partial task execution whenever it is possible to limit extra bandwidth consumption and edge computation costs. Moreover, with the increase in the number of vehicles, the cost curve of LEVEC increases slowly, which further demonstrates that its context-aware reasoning mechanism effectively balances task urgency and economic performance. This resulting incremental cost shows that LEVEC is able to scale effectively without introducing an excessive monetary overhead under heavy system load.

MAPTD3 has a bigger average cost than LEVEC for any number of vehicles, and the difference becomes more significant with the increase in the number of vehicles. Although MAPTD3 learns the optimal resource allocation through deep reinforcement learning, its cost function shows the preference for aggressive offloading to edge servers in congested environments as well. That leads to higher bandwidth usage and

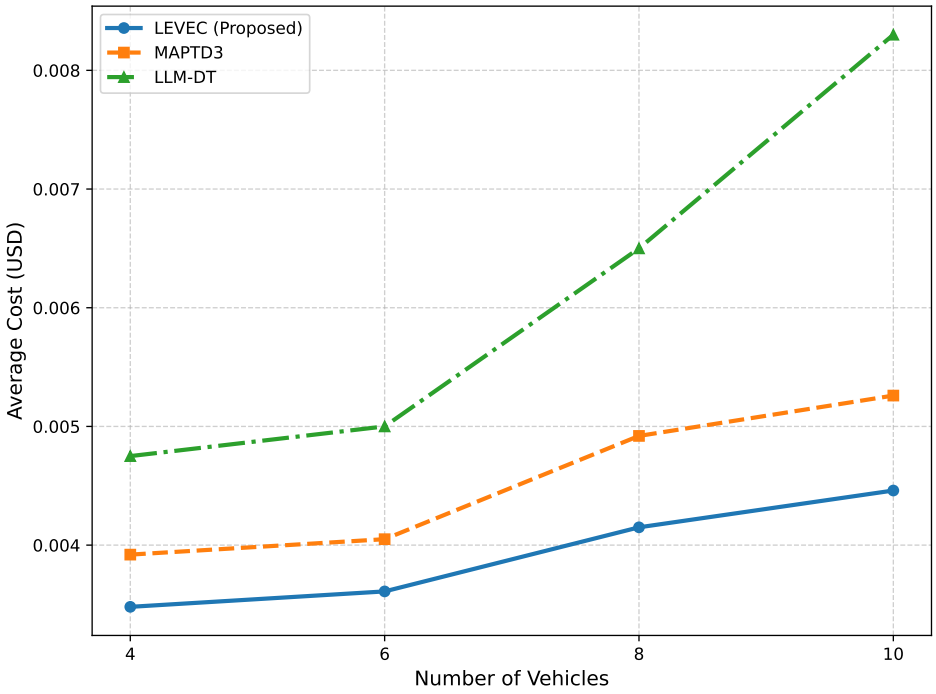



Figure 4.5: Average cost comparison on varying vehicle density

computational price. The increased cost difference under a larger vehicle count between MAPTD3 and LEVEC further suggests the limitations of reward-guided optimization which lacks semantic knowledge of task significance and execution context.

The cost analysis reveals that the our proposed LEVEC framework achieves a notably lower operational cost in comparison to the MAPTD3 and LLM-DT approaches across increasing vehicular densities. Specifically, LEVEC exhibits an average cost reduction of approximately 12.6% relative to MAPTD3 and 35.5% relative to LLM-DT over the tested vehicle counts of 4 to 10. While all methods display a general upward trend in cost with scaling system size, LEVEC maintains the most economical profile, demonstrating both efficiency and scalability in resource allocation. The progressively widening cost gap, particularly against LLM-DT, underscores LEVEC's optimized task scheduling and reduced reliance on high-overhead communication mechanisms, affirming its practical viability for cost-sensitive edge computing deployments in vehicular networks.

In summary, the results solidly prove that LEVEC is the most cost-effective of all methods. With the combination of lightweight LLM reasoning and adaptative task offloading and execution strategies, LEVEC balances intelligence together with economic efficiency. The comparison further validates that minimizing (unneeded) offloading and avoiding excessive coordination overhead are critical in achieving low operational cost for vehicular edge computing systems. The above benefits are amplified by the increasing number of vehicles, this again serves to support the feasibility and scalability

of proposed LEVEC scheme.

## 4.5 Conclusion

This chapter presented a performance analysis of the proposed LEVEC framework, comparing it with MAPTD3 and LLM-DT in terms of reasoning ability, latency, and operational cost under varying vehicular densities. Reasoning experiments established Llama 3-8B as the preferred model due to its consistent and context-aware task prioritization and system weight optimization. Simulation results confirmed that LEVEC achieves significantly lower latency and cost across all tested scenarios.

The performance gains are attributed to LEVEC's adaptive, context-aware scheduling and efficient offloading strategy, which reduce overhead and queuing delays. In contrast, MAPTD3 exhibited slower convergence under dynamic conditions, while LLM-DT incurred high synchronization and computational costs. Overall, LEVEC demonstrates a robust balance of intelligence, responsiveness, and cost-efficiency, proving its viability as a scalable solution for next-generation vehicular edge computing.

# 5. Conclusion

In this chapter, we present our concluding thoughts on the proposed LEVEC framework. We also address the constraints encountered by our system and highlight future directions and aspirations for further exploration in this area of research.

## 5.1 Summary of the Study

Our thesis introduced LEVEC, which is an LLM-enhanced task offloading and execution framework tailored for solving the low latency optimization problem while minimizing the network cost within a vehicular edge computing scenario. LEVEC is able to efficiently to handle heavy and heterogeneous computation vehicular tasks in a dynamically changing traffic environment by introducing context-aware reasoning into the offloading decision-making. The proposed framework combines LLM-based priority scoring and adaptive execution for balancing local processing, partial offloading, and full offloading on RSUs to minimize the end-to-end system latency and cost while retaining delay sensitivity of time-critical applications.

LEVEC was evaluated by extensive simulations under different vehicle densities and compared with state-of-the-art methods such as MAPTD3 and LLM-DT. The experimental results show that LEVEC significantly outperforms the aforementioned baselines in average latency and average cost, especially when the vehicular density is high. This efficiency is obtained by minimizing the unnecessary offloading, cutting down queuing delays and integrating task urgency (or context) directly into decision-making for scheduling and resource assignment. In contrast to those based reinforcement learning approaches which only use numerical state representations, or digital twin (which introduce further system overhead), LEVEC can well balance the intelligence and efficiency of execution.

The main contributions of the work are LLM-based contextual reasoning for task prioritization, enabling support for partial and hybrid task offloading in a vehicular edge computing context, and development of an optimization model that recalibrates itself dynamically with varying network and mobility conditions. The developed LEVEC framework is scalable, robust, and efficient enough to be applied in real-world scenarios such as intelligent transportation systems, autonomous driving assistance,

and safety-critical vehicular services. In summary, we believe our study presents promising and lightweight LLM-based decision-making strategies for designing next-generation vehicular edge computing systems.

## 5.2 Limitations and Future Work

Although our LEVEC framework has shown remarkable progress in offloading tasks and system efficiency, there are some limitations that can be improved for future work.

- **Incorporation of Specialized LLM:** LEVEC uses a general-purpose LLM, which has broader task prioritization and does not have enough specialized reasoning for applications that are domain or task specific. A possible direction for future work would be to investigate incorporating domain-specific LLM models that are fine-tuned on automotive data. This may enable the framework to be more responsive to specific requirements of certain automotive applications, such as autonomous driving or real-time navigation.

- **Implementation of Full LLM Model Than Quantized LLM:** LEVEC performs inference with a quantized LLM model so as to save processing cost and reduce inference latency. But maybe, if the full LLM is adopted, it will help improve the reasoning of the system for large-scale VEC network scenarios when there are more vehicles to be used and/or tasks to be processed. Future research could study the computational complexity vs. model accuracy trade-offs, bearing in mind the incremental building and resource constraints of the edge infrastructure.

- **Improving Prompt Engineering Even Further:** The current approach is based on manual prompt engineering that specifies the task-offloading decisions and priority scoring. For better efficiency, extended research might consider advanced prompt engineering techniques to dynamically adjust and optimize decision-making so that LEVEC is able to further manage the different and variant real-time contexts. The system could also improve performance under diverse operating conditions by turning to self-optimization for the model or prompt modification with context.

- **Introducing Dynamic Dataset:** The dataset in our study only characterizes the task generation for one time slot and fails to capture long-term fluctuations of vehicular tasks. On the other hand, a dynamic dataset that emulates tasks during different time slots could make LEVEC easily adapt to the variable network and tackle temporal correlation better. They could allow the system to improve with time, leading to more precise and scalable offloading in practical situations.

The limitations we identify and the future work directions suggest a roadmap to improve the LEVEC framework, making it more adaptive, scalable, and efficient to advance vehicular edge computing systems.

# Reference


[1] J. Zhao, H. Quan, P. Ge, Y. Huang, and Y. Xiao, "Vehicular edge computing system: A survey," *IEEE Internet of Things Magazine*, vol. 8, no. 6, pp. 122–128, 2025.

[2] M. A. K. Raiaan, M. S. H. Mukta, K. Fatema, N. M. Fahad, S. Sakib, M. M. J. Mim, J. Ahmad, M. E. Ali, and S. Azam, "A review on large language models: Architectures, applications, taxonomies, open issues and challenges," *IEEE Access*, vol. 12, pp. 26839–26874, 2024.

[3] "Large language model market size, share and trends 2026 to 2035." https://www.precedenceresearch.com/large-language-model-market. [Online; accessed 15-January-2026].

[4] K. Cao, Y. Liu, G. Meng, and Q. Sun, "An overview on edge computing research," *IEEE Access*, vol. 8, pp. 85714–85728, 2020.

[5] A. Nimodiya and S. Ajankar, "A review on internet of things," *International Journal of Advanced Research in Science, Communication and Technology*, pp. 135–144, 01 2022.

[6] R. Meneguette, R. De Grande, J. Ueyama, G. P. R. Filho, and E. Madeira, "Vehicular edge computing: Architecture, resource management, security, and challenges," *ACM Comput. Surv.*, vol. 55, Nov. 2021.

[7] R. Meneguette, R. De Grande, J. Ueyama, G. P. R. Filho, and E. Madeira, "Vehicular edge computing: Architecture, resource management, security, and challenges," *ACM Comput. Surv.*, vol. 55, Nov. 2021.

[8] Z. Jin, C. Zhang, G. Zhao, Y. Jin, and L. Zhang, "A context-aware task offloading scheme in collaborative vehicular edge computing systems," *KSII Transactions on Internet and Information Systems*, vol. 15, pp. 383–403, 02 2021.

[9] C. Sandu and I. Susnea, "Edge computing for autonomous vehicles. a scoping review.," pp. 1–5, 11 2021.

[10] Sahil and S. K. Sood, "Smart vehicular traffic management: An edge cloud centric iot based framework," *Internet of Things*, vol. 14, p. 100140, 2021.

[11] P. Rosayyan, J. Paul, S. Subramaniam, and S. I. Ganesan, "An optimal control strategy for emergency vehicle priority system in smart cities using edge computing and iot sensors," *Measurement: Sensors*, vol. 26, p. 100697, 2023.

[12] A. Carvalho, N. O' Mahony, L. Krpalkova, S. Campbell, J. Walsh, and P. Doody, "Edge computing applied to industrial machines," *Procedia Manufacturing*, vol. 38, pp. 178–185, 01 2019.

[13] Y.-A. Daraghmi, "Vehicle speed monitoring system based on edge computing," 02 2022.

[14] X. Yuan and H. Li, "Llm-driven offloading decisions for edge object detection in smart city deployments," *Smart Cities*, vol. 8, no. 5, 2025.

[15] S. Wang, X. Song, H. Xu, T. Song, G. Zhang, and Y. Yang, "Joint offloading decision and resource allocation in vehicular edge computing networks," *Digital Communications and Networks*, 2023.

[16] J. Liu, Y. Zou, G. Du, X. Zhang, and J. Wu, "Task offloading and resource allocation for icvs in vehicular edge computing networks based on hybrid hierarchical deep reinforcement learning," *Sensors*, 2025.

[17] B. Li, L. Zhu, and L. Tan, "Optimizing task offloading and resource allocation in latency-constrained vehicular edge computing," *Cluster Computing*, 2025.

[18] A. Tyagi and N. Sreenath, *Intelligent Transportation System: Past, Present, and Future*, pp. 23–47. 11 2022.

[19] J. Zhao, H. Quan, M. Xia, and D. Wang, "Adaptive resource allocation for mobile edge computing in internet of vehicles: A deep reinforcement learning approach," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 4, pp. 5834–5848, 2024.

[20] Y. Cui, H. Shi, R. Wang, P. He, D. Wu, and X. Huang, "Multi-agent reinforcement learning for slicing resource allocation in vehicular networks," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 2, pp. 2005–2016, 2024.

[21] Z. Abbas, S. Xu, and X. Zhang, "An efficient partial task offloading and resource allocation scheme for vehicular edge computing in a dynamic environment," *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, no. 2, pp. 2488–2502, 2025.

[22] L. Liu, J. Feng, X. Mu, Q. Pei, D. Lan, and M. Xiao, "Asynchronous deep reinforcement learning for collaborative task computing and on-demand resource allocation in vehicular edge computing," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 12, pp. 15513–15526, 2023.

[23] S. Murshed, M. A. Razzaque, P. Roy, and M. M. Hassan, "Dynamic task prioritization and fair resource allocation in vehicular edge computing," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pp. 1–6, 2025.

[24] Q. Wu, Y. Xie, P. Fan, D. Qin, K. Wang, N. Cheng, and K. B. Letaief, "Large language model-based task offloading and resource allocation for digital twin edge computing networks," *IEEE Transactions on Intelligent Transportation Systems*, vol. 13, no. 10, pp. 12241–12252, 2025.

# List of Acronyms

| | |
|---|---|
| LEVEC | LLM Enhanced Vehicular Edge Computing Framework |
| LLM | Large Language Model |
| CAGR | Cumulative Annual Growth Rate |
| VEC | Vehicular Edge Computing |
| RSU | Roadside Unit |
| DRL | Deep Reinforcement Learning |
| LLM-DT | Large Language Model with Digital Twin |
| DT | Digital Twin |
| MEC | Mobile Edge Computing |
| RL | Reinforcement Learning |
| DRL | Deep Reinforcement Learning |
| V2I | Vehicle-to-Infrastructure |
| IoV | Internet of Vehicles |
| IoT | Internet of Things |
| GPU | Graphical Processing Unit |
| A3C | Asynchronous Advantage Actor-Critic |
| MAPTD3 | Multi-Agent Parallel Twin Delayed Deep Deterministic Policy Gradient |

*Please use the following page format for the hard-cover binding of your report.*

# LLM Enhanced Task Offloading and Execution Framework in Vehicular Edge Computing

*by*

**Faysal Hossain Tomal - 221002220**
**Afzal Hossain - 221002628**
**K M Monoarul Islam Shovon - 221002213**

*Thesis for the degree of*

**Bachelor of Science in Computer Science and Engineering**


**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**GREEN UNIVERSITY OF BANGLADESH**

**FALL 2025**